Kurdistan Regional Government – Iraq
Ministry of Higher Education and Scientific Research
University of Duhok
College of Science
Department of Computer Science

Evaluating Large Language Models (LLMs) for Code Generation for Internet of Things (IoT) Constrained Devices

A Thesis

Submitted to the Council of the College of Science/University of Duhok in Partial Fulfilment of the Requirements for the Degree of Master (MSc) in Computer Science

By
Sardar Kaukaz Jabrw Ali
BSc. in Computer Science, University of Duhok – 2019

Supervised By
Assistant Professor Dr. Qusay Idrees Sarhan
University of Duhok, Duhok, Kurdistan Region, Iraq

1447 A.H. 2026 A.D. 2726 K.

Acknowledgements

I express my gratitude and respect to my supervisor Assistant Professor Dr. Qusay Idrees Sarhan for consistently providing constructive feedback, and supporting me during my master's research study.

I am also thankful to the University of Duhok, especially, the College of Science and the Department of Computer Science for their support in various forms, for providing the resources needed, and for making it possible for me to conduct this research.

Much thanks go to my family for their support, perseverance, and understanding, as well as to my friends, colleagues, and all other people who provided me with technical and administrative help, as well as those who assisted with proofreading, editing, and other essential tasks.

Sardar Kaukaz Jabrw Ali

May 2026

Dedication

To my family and friends, thank you for standing by me every step of the way. This achievement belongs to you as much as it does to me.

Sardar Kaukaz Jabrw Ali

May 2026

List of Publications

This thesis has resulted in the following peer-reviewed publications:

1. S. K. Jabrw and Q. I. Sarhan, **"A Systematic Survey on Large Language Models for Code Generation**", ARO, vol. 13, no. 2, pp. 83–99, Aug. 2025. https://doi.org/10.14500/aro.12159
   Impact Factor: 2.1 (as of 2025).

2. S. K. Jabrw and Q. I. Sarhan, "**Evaluating Large Language Models (LLMs) for Arduino Code Generation**", ARO, vol. 14, no. 1, pp. 37–47, Jan. 2026. https://doi.org/10.14500/aro.12344
   Impact Factor: 2.1 (as of 2025).

Abstract

Large Language Models (LLMs), a type of generative Artificial Intelligence (AI), have transformed code generation by converting natural language prompts into executable code. Yet, their capabilities in generating code for Internet of Things (IoT) resource-constrained devices such as Arduino, remained underexplored. This thesis evaluates six state-of-the-art LLMs (i.e., ChatGPT-4o, Gemini 2.0 Flash, DeepSeek-V3, Claude 3.5 Sonnet, GitHub Copilot, and LLaMA-3) for generating correct, efficient, and high-quality Arduino code. The evaluation was performed across six dimensions, namely functional correctness, runtime efficiency, memory usage, code quality, similarity to human-written code, and multi-round error correction. The results reveal that ChatGPT-4o achieves the highest functional correctness and aligns closely with human-written code in readability and similarity. Gemini 2.0 Flash generates codes that run faster. However, its generated codes come with high complexity and low similarity compared to the reference codes. The codes generated by DeepSeek-V3 are good in terms of Flash memory optimization. Claude 3.5 Sonnet poses issues with prompt understanding and the correctness of generated code improves by using multi-round error correction. Overall, the results obtained show that no single LLM performs better in all the conducted evaluations. In other words, different LLMs perform better in different code generation aspects. Hence, for programs with different priorities, different LLMs should be chosen. For example, ChatGPT-4o excels in functional correctness (96.8%), while Gemini 2.0 Flash in execution time, and DeepSeek-V3 in memory efficiency. These findings contribute to the ongoing development of LLMs by identifying the strengths and weaknesses of each model in generating Arduino code, where Arduino is a widely used platform for IoT and embedded systems.

Keywords: Large Language Models, Arduino, Code Generation, Internet of Things, Code Performance.

Table of Contents

Supervisor Certification ..... **i**
Declaration and Recommendation of Linguistic Evaluator ..... **ii**
Recommendation of the Head of the Academic Department ..... **iii**
MSc Examination Committee Certification ..... **iv**
Declaration ..... **v**
Acknowledgements ..... **vi**
Dedication ..... **vii**
List of Publications ..... **viii**
Abstract ..... **ix**
Table of Contents ..... **x**
List of Figures ..... **xiii**
List of Tables ..... **xiv**
List of Abbreviations ..... **xv**
Chapter 1: Introduction **1**
1.1 Overview and Background 1
1.2 Problem Statement 3
1.3 Aim and Objectives 3
1.4 Contributions 4
1.5 Limitations 4
1.6 Thesis Outline 5
Chapter 2: Literature Survey **6**
2.1 Introduction 6
2.2 Systematic Literature Survey 6
2.2.1 Methodology 6
2.2.2 Identification of Research Objectives and Questions 7
2.2.3 Search Strategy 7
2.2.4 Paper Selection 8
2.2.5 Data Extraction and Analysis 9
2.2.6 Documentation 9
2.2.7 Literature Analysis 9
2.2.7.1 Programming Languages for Evaluation (RQ1) 9

2.2.7.2 Metrics for Code Quality (RQ2) 10
2.2.7.3 Use Cases and Applications (RQ3) 13
2.2.7.4 Prompt Design and Effectiveness (RQ4) 14
2.2.7.5 Security of LLM-Generated Code (RQ5) 15
2.2.7.6 Benchmark/Dataset Characteristics (RQ6) 16
2.2.7.7 Models used in Evaluations (RQ7) 18
2.2.7.8 Code Analysis Tools used in Evaluations (RQ8) 20
2.2.7.9 Evaluation Challenges (RQ9) 21
2.2.7.10 Future Research Directions (RQ10) 23
2.3 Comparison with Related Studies 24
Chapter 3: Research Methodology 29
3.1 Introduction 29
3.2 Research Methodology 29
3.3 Research Questions 30
3.4 Used Dataset 31
3.5 Used LLMs 31
3.6 Used Prompt Type 31
3.7 Used Evaluation Metrics 32
Chapter 4: Results and Discussion 36
4.1 Introduction 36
4.2 Overall Correctness (RQ1) 36
4.3 Code Performance (RQ2) 37
4.4 Multi-attempt Code Correction (RQ3) 39
4.5 Code Complexity (RQ4) 40
4.6 Code Similarity (RQ5) 41
4.7 Discussion 42
Chapter 5: Conclusions and Future Works 45
5.1 Conclusions 45
5.2 Future Works 45

References..... **46**
الخلاصة:..... **56**
پۆختە..... **58**

List of Figures

Figure 2.1: Five-stage systematic survey methodology. 6

Figure 2.2: Outcomes of paper selection process.. 9

Figure 3.1: The procedure used to evaluate LLMs 29

Figure 3.2: Example of prompt and code solution generated by ChatGPT. 31

Figure 4.1: Overall correctness of LLM-generated code……..……..…….....35

Figure 4.2: Execution time performance of LLM-generated code. 36

Figure 4.3: Flash memory usage performance of LLM-generated code. 37

Figure 4.4: SRAM usage performance of LLM-generated code. 38

Figure 4.5: Multi-attempt code fixing progress across LLMs 39

Figure 4.6: Cyclomatic complexity of LLM-generated code. 39

Figure 4.7: NCLOC of LLM-generated code. 40

Figure 4.8: CodeBLEU scores of LLM-generated code. 41

List of Tables

Table 2.1: Most used programming languages for LLMs evaluation. 10

Table 2.2: Evaluation metrics for LLM-generated code 12

Table 2.3: Summary of LLM usage categories and actions 14

Table 2.4: Common prompts strategies 15

Table 2.5: Benchmarks used for evaluating code generation in LLMs 17

Table 2.6: LLMs used for code generation evaluation 19

Table 2.7: Code analysis tools used for evaluating LLM-generated code 21

Table 2.8: Summary of comparison with related studies……..…...……..27

Table 3.1: Arduino IDE specifications……34

Table 4.1: Summary of evaluting LLMs for Arduino code generation 43

List of Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
ASE	Automated Software Engineering
AST	Abstract Syntax Tree
AWS	Amazon Web Services
BLEU	Bilingual Evaluation Understudy
CC	Cyclomatic Complexity
CEMs	Code Evaluation Metrics
CodeBLEU	Code Bilingual Evaluation Understudy
CoT	Chain-of-Thought
CPU	Central Processing Unit
CWE	Common Weakness Enumeration
GB	Gigabyte
GPU	Graphics Processing Unit
HDL	Hardware Description Language
IDE	Integrated Development Environment
IoT	Internet of Things
LLM	Large Language Model
LOC	Lines of Code
MBPP	Mostly Basic Python Problems
MCU	MiCrocontroller Unit
NCLOC	Non-Comment Lines of Code
NLP	Natural Language Processing
Pass@k	Pass at *k* Metric
RQ	Research Question
SCoT	Structured Chain-of-Thought
SDK	Software Development Kit
SE	Software Engineering
SLR	Systematic Literature Review
SoC	System on a Chip
SRAM	Static Random Access Memory
T2C	Text-to-Code

Chapter 1: Introduction

Chapter 1: Introduction

1.1 Overview and Background

Large Language Models (LLMs) are deep learning models, typically comprising billions of parameters, trained on massive datasets of unlabeled text using self-supervised or semi-supervised learning techniques [1]. While early generative models utilized Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, the field was revolutionized by the introduction of the Transformer architecture [2]. Transformers utilize self-attention mechanisms to weigh the importance of different words in an input sequence, allowing them to capture dependencies and context more effectively than their predecessors [3]. LLMs have profoundly transformed various professional fields, extending their capabilities far beyond traditional Natural Language Processing (NLP) tasks [1]. This paradigm shift is especially evident in software engineering, where LLMs are increasingly recognized for their potential to enhance productivity and automate tasks [4].

LLMs are trained on large corpora of natural language (e.g., articles) and programming code (e.g., GitHub repositories), enabling these models to convert prompts written in natural language into not only executable, functional code but beyond code generation, where it can assist developer in debugging [5], optimization [6] and writing documentations [7], which makes them an increasingly relevant tool for developers and researchers. LLMs have made strides in academia and industry, transforming the way software is maintained and developed since the release of ChatGPT in late 2022 [2].

Despite their promise in assisting developers with code generation, challenges remain, as evaluating the quality of the generated code is important for understanding its usefulness and limitations in practice [8]. Since these models are trained on data from different sources, their reliability and consistency cannot be guaranteed. Therefore, the quality of generated code must be evaluated in terms of maintainability, functionality, and performance to know its effectiveness in real-world applications [3]. Furthermore, LLMs can generate code with security vulnerabilities and logical errors, just as in human-written code, yet the models can still be confident that the code is correct, despite these errors [9]. These limitations have raised concerns about evaluating the strength and limitation of LLMs in code-generation tasks.

To assess the quality of automatically generated code, researchers typically rely on Code Evaluation Metrics (CEMs), which can be broadly divided into two categories, match-based metrics and execution-based metrics [10]. Match-based metrics evaluate the similarity between a generated code and a reference (ground-truth) solution. The comparison is generally based on textual or structural overlaps. A widely used metric in this category is Bilingual Evaluation

Understudy (BLEU). Originally developed for machine translation evaluation, BLEU measures the degree of n-gram overlap between generated and reference texts. Execution-based metrics evaluate the functional correctness of generated code by running it against a set of predefined test cases (i.e., inputs). A well-known metric in this category is Pass@k, which measures the percentage of programming tasks for which at least one of k code samples produced by the LLM passes all the provided test cases.

The Internet of Things (IoT) refers to a network of interconnected physical devices embedded with sensors, actuators, and communication modules that enable data collection and exchange over the Internet. IoT systems typically consist of four main components: physical devices (e.g., microcontrollers and sensors), communication technologies (e.g., WiFi) for data transmission, data processing units such as cloud or edge computing platforms, and user interfaces for monitoring and controlling. IoT systems are widely used in various domains, including smart homes, healthcare, agriculture, industrial automation, and environmental monitoring. These systems typically rely on resource-constrained devices (e.g., Arduino and Raspberry Pi) that operate with limited processing power, memory, and energy resources []. While LLMs show promising results in general-purpose programming languages such as Python, their effectiveness in generating code for IoT resource-constrained devices such as Arduino remains underexplored. Arduino is a cheap, low-power, and small-sized computing device [11]. Arduino has been used for over a decade in a wide range of systems and applications such as IoT, robotics, smart homes, and real-time control and monitoring systems [12]. Currently, Arduino is one of the most popular development platforms for IoT due to its simplicity, flexibility, affordability, and large community support [13]. However, Arduino devices have limited computing resources. Therefore, it is crucial to efficiently code such devices to reduce execution time and energy consumption, which are an essential factor when it comes to battery-powered devices, such as those used in IoT and wireless sensor networks. Finally, when using efficient programs, developers can add more features or include more functionalities without exceeding device constraints.

The following example demonstrates how a developer can prompt an LLM, such as ChatGPT to generate a code for Arduino:

"Write an Arduino Uno Rev3 code that prints "Task", waits 100ms, then prints "Task" again."

The following output is expected from ChatGPT:

```
void setup()
{
  Serial.begin(9600);
  Serial.println("Task");
  delay(100);
  Serial.println("Task");
}
void loop()
{
}
```

To better understand the capabilities and limitations of LLMs for code generation, this thesis evaluates and investigates their capabilities in generating correct, efficient, and high-quality Arduino code. To this end, six state-of-the-art LLMs (i.e., 6, Gemini 2.0 Flash, DeepSeek-V3, Claude 3.5 Sonnet, GitHub Copilot, and LLaMA-3) were evaluated using a dataset of 31 Arduino subject programs covering various aspects of coding and a set of evaluation metrics, offering a comprehensive assessment of LLMs' capabilities for Arduino code generation.

1.2 Problem Statement

While LLMs have demonstrated promising performance in generating code for general-purpose programming languages such as Python and Java, their effectiveness in programming IoT resource-constrained devices like Arduino remains unexplored. Although Arduino is widely used in IoT, robotics, and real-time monitoring applications, prior studies have not systematically explored whether LLMs generate correct, efficient, and optimized code specifically for Arduino IoT constrained device. This is important because Arduino has limited computational resources and memory space, therefore it requires efficient code to maintain reasonable responsiveness and allow adding additional functionality without exceeding hardware limitations.

1.3 Aim and Objectives

The aim of this thesis is to assess how well different state-of-the-art LLMs can generate Arduino code. More precisely, it aims to evaluate the quality and performance of the generated code and discover the strengths and weaknesses of these LLMs when applied to resource-constrained devices. In order to accomplish this, the following objectives are established for:

- To evaluate the correctness, efficiency, and performance of Arduino code generated by different state-of-the-art LLMs.
- To introduce a dataset of Arduino programs with various coding tasks as a benchmark

for evaluation.

- To compare the strengths and weaknesses of different LLMs for Arduino code generation.

1.4 Contributions

This thesis achieves several contributions, as follows:

- It evaluates six LLMs in generating Arduino code, using different evaluation metrics to assess the LLMs' performance in generating the correct, efficient, and high-quality code for Arduino.
- It introduces a dataset of 31 Arduino programs with various coding tasks that is used to evaluate the LLMs with diverse coding aspects, focusing on code performance.
- It reveals the strengths and limitations of current LLM capabilities in Arduino programming. This thesis contributes to a deeper understanding of AI-assisted code generation.

1.5 Limitations

Although this thesis contributes to understanding the capabilities of LLMs in Arduino code generation, it has several limitations that should be acknowledged, as follows:

- Only six state-of-the-art LLMs were evaluated. Thus, the findings cannot be generalized to other LLMs that were not used, nor to any future versions of the same used LLMs. However, the used LLMs were chosen with care for their popularity and the recent developments in the domain.
- The LLMs were evaluated only as tools via the user interfaces that are publicly available online, without any fine-tuning or control over internal configurations, acknowledging that their non-deterministic nature may give different results on different runs.
- This thesis uses only one prompting strategy, which is zero-shot prompting. Other prompting strategies such as chain-of-thought and few-shot were not explored and may influence the quality of generated codes.
- The experiments were conducted using a dataset of 31 Arduino programs. While Arduino programming language is a widely used language in IoT, the results cannot be generalized to other languages.

1.6 Thesis Outline

The result of this thesis is organized into following chapters:

- **Chapter Two** provides a systematic literature review of LLMs evaluation in code generation, emphasizing evaluation methods, target programming languages, benchmarks, evaluation metrics, and gaps.
- **Chapter Three** describes the research methodology, dataset, and LLMs employed in the experimental part of this thesis. It also includes the research questions that were set to evaluate the used LLMs alongside many directions.
- **Chapter Four** presents the experimental results of this thesis and their discussion by addressing each research question.
- **Chapter Five** presents the conclusions of this thesis and provides suggestions for future works.

Chapter 2:
Literature Review

Chapter 2: Literature Survey

2.1 Introduction

LLMs have transformed the way software is maintained and developed, where they have offered powerful tools to automate the software development process. Despite these promises in assisting developers with tasks such as code generation, challenges remain in evaluating the quality, security, and effectiveness of LLM-generated code. Therefore, a literature review of available studies on LLM code generation is necessary. This chapter presents the results of a systematic literature survey on LLMs for code generation, such as the most frequently used programming languages, the metrics used, and the scenarios in which developers apply them across the software development life cycle. Ultimately, this survey helps to know how researchers use and evaluate LLM-generated code and helps identify research gaps in literature.

2.2 Systematic Literature Survey

2.2.1 Methodology

Inspired by [14], the methodology used for performing this survey comprises five stages as shown in Figure 2.1. The first stage is identifying the survey's objectives and RQs. Secondly, a strategy is defined to identify relevant publications related to thesis' topic. Thirdly, the selection and filtering of the publications obtained in the previous stage are carried out. Next, the fourth stage is data extraction, where the relevant publications are reviewed and key information required to answer the identified RQs is extracted. The final stage includes reporting and documenting the results. The details of these five stages are presented in the following sections.

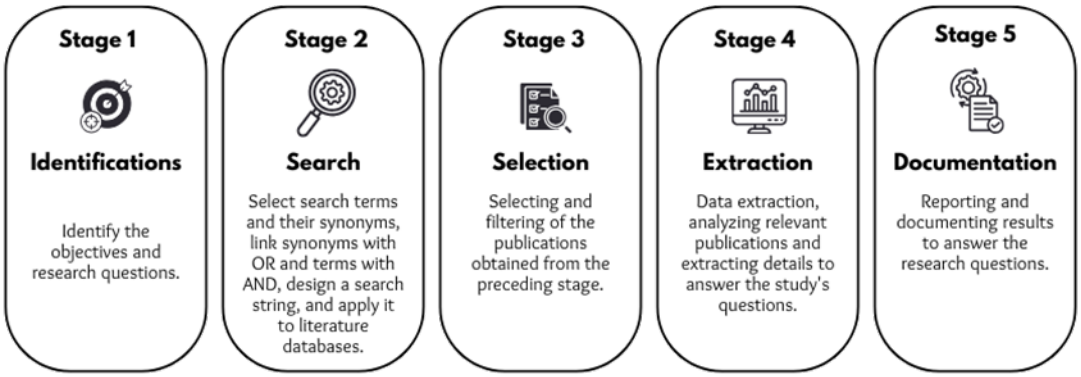

Figure 2.1: Five-stage systematic survey methodology.

2.2.2 Identification of Research Objectives and Questions

A. Research Objectives

The objectives of this literature survey are to analyze studies published on using LLMs for code generation and to identify, review, and categorize state-of-the-art contributions in the field.

B. Research Questions

This survey has identified and addressed several RQs, each of which refers to a specific facet of the topic, as outlined below:

- **RQ1:** Which programming languages are used in evaluating LLMs performance for code generation tasks?
- **RQ2:** Which metrics are most frequently used to evaluate the quality of LLM-generated code?
- **RQ3:** For which programming scenarios, duties, and objectives are developers using LLMs?
- **RQ4:** How do different prompts impact the effectiveness of LLMs in code generation tasks?
- **RQ5:** Is the code generated by LLMs secure?
- **RQ6:** What are the characteristics of benchmarks used for evaluating the performance of LLMs in code generation tasks?
- **RQ7:** Which LLMs are used in the evaluation of code generation?
- **RQ8:** Which code analysis tools are used to evaluate code generated by LLMs?
- **RQ9:** What are the challenges in evaluating LLMs for code generation?
- **RQ10:** What are the potential future research directions for using LLMs for code generation?

2.2.3 Search Strategy

A. Literature Sources

Well-known online databases, such as IEEE Xplore, Elsevier Science Direct, ACM Digital Library, etc., indexing publications relevant to the scope of this thesis, were selected as literature sources. Each database was chosen for its high-quality, peer-reviewed studies in computer science, and technology, making it ideal for this thesis.

B. Search String

The following search string was used to identify publications relevant to this survey within the literature sources:

"(Code Generation) AND (LLM OR Large Language Model OR Generative AI)"

All search terms were linked using Boolean operators. "OR" connected synonyms or related terms, while "AND" linked the main terms.

2.2.4 Paper Selection

A set of inclusion and exclusion criteria were established and employed to decide whether a publication is relevant to this thesis or not. The criteria listed below have been applied based on the titles, abstracts, and full-text readings. Figure 2.2 illustrates the number of papers included or excluded based on these criteria, where 74 papers were included in this survey.

Inclusion criteria:

- Publications related directly to evaluating LLMs for code generation. These studies were selected because they evaluate the quality of the generated code.
- Publications published online over the past four years (2021 to 2024). According to the search and exploration of the literature, publications on evaluating the code generated by LLMs started in 2021, with the study [15] being the first to evaluate LLMs for code generation.

Exclusion criteria:

- Publications that are not published in the English language.
- Publications that are not directly related to the research topic.
- Publications that are not peer-reviewed (e.g., gray literature).
- Publications that are not published electronically.
- Duplicated publications.
- Publications without empirical results.

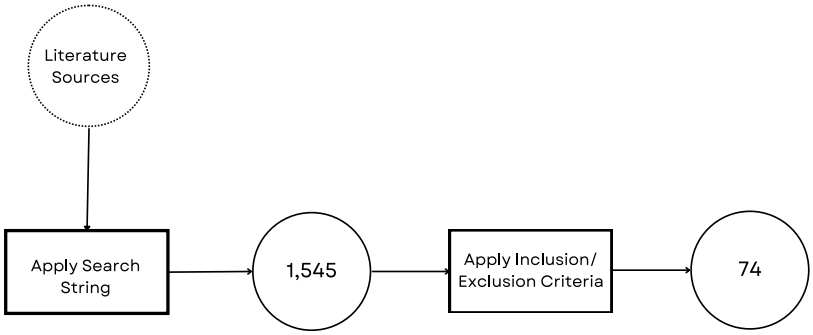

Figure 2.2: Outcomes of paper selection process.

2.2.5 Data Extraction and Analysis

From the selected papers, data were extracted and extensively analyzed to answer the identified RQs. The data were extracted first and then double-checked to ensure accuracy.

2.2.6 Documentation

The data extracted and analyzed from the previous step are documented and presented in this final stage to address the RQs.

2.2.7 Literature Analysis

To answer the identified RQs of this survey, all the selected publications were thoroughly analyzed. Each RQ, represented by a short title, is discussed in the following subsections based on the obtained results.

2.2.7.1 Programming Languages for Evaluation (RQ1)

All the selected papers of this survey were analyzed to determine which programming languages are used as targets to evaluate the quality of the generated code. The majority of studies were conducted across three languages: 31 on Python, 18 on Java, and 12 on C++. Additionally, some of them studied several languages to highlight LLMs' flexibility. Very few papers evaluated code generated for hardware programming languages: just 4 on Verilog, 2 on VHDL, and 1 on Arduino. Table 2.1 shows that most studies were conducted on Python, which is one of the most widely adopted programming languages among developers and researchers, it provides libraries and frameworks for a wide range of applications [16]. There is also plenty of Python code on GitHub and other public repositories for training and testing LLMs. In addition, popular datasets such as HumanEval and Mostly Basic Python Problems (MBPP) rely primarily on Python because it is dominant among programming languages. Its prominence in education and research shows that it is familiar for research, which makes it a preferred choice for conducting research.

Table 2.1: Most used programming languages for LLMs evaluation.

#	Programming Languages	Published Papers	Total
1	Python	[4], [17], [16], [1], [18],[19],[9],[20],[21],[22],[23],[24],[25],[8],[26],[27], [28], [29],[30],[31], [3],[32],[33],[34] , [35],[36],[37],[38],[39],[40],[41]	31
2	Java	[18],[22],[24],[27],[42],[43],[29],[44],[3],[32],[45],[33],[34] ,[46],[47],[38],[40], [48]	18
3	C++	[24],[49],[26],[27],[29],[3],[47],[34],[39],[50],[40],[48]	12
4	C	[9],[21],[24],[27],[29],[3],[50]	7
5	Verilog	[51],[52],[52],[53]	4
6	JavaScript	[3],[54],[38]	3
7	VHDL	[51],[55]	2
8	R	[56]	2
9	Arduino	[57]	1
10	Go	[27],[34],[38]	3
11	Assembly Language (32)	[58]	1
12	HTML	[29]	1

2.2.7.2 Metrics for Code Quality (RQ2)

The survey findings reveal that a variety of evaluation metrics are used by researchers to evaluate the quality of generated code. Most of these studies have included metrics across different categories (e.g., code complexity and performance), indicating an intention to combine evaluations in a more comprehensive way. Specifically, some of the most commonly used metrics are functional correctness (e.g., pass ratio/pass@k and accuracy rate), which evaluate the correctness of generated code. Similarity metrics such as Bilingual Evaluation Understudy (BLEU) and CodeBLEU, which measure the similarity between generated and developer-written code, are often used. Likewise, measuring Cyclomatic Complexity (CC) alongside counting Lines of Code (LOC) were used to evaluate the logical and structural complexity of the code. Additionally, time and space complexity were also used together to assess the performance of code in terms of execution speed and memory usage. Moreover, generation speed and average completion time were used as indicators of LLMs' responsiveness to real-world demands and often used as usability metrics.

All reviewed studies indicate that none of the used LLMs can perform best in all task types. Consequently, no single useful and successful evaluation metric has been explicitly identified. After examining the best metrics for evaluating the generated code, it is clear that no single metric is best, and that robust evaluation requires a multifaceted approach.

Functional correctness metrics, such as pass ratio/pass@k, measure whether generated code passes a set of predefined test cases or not. These metrics are easier and faster to measure because they can be automatically measured rather than manually implementing each test case. However, pass ratio/pass@k has limitations, such as reliance on the number of predefined test cases, which can be inadequate, making them insufficiently accurate and potentially leading to false opinions. Another notable limitation is that developers do not run the LLM multiple times (e.g., 100 times) to generate a single piece of code, so pass@k is not representative of real-

world usability. Although it shows the randomness of LLM output, it still does not align with how users interact with an LLM to write code, which may require many attempts and prompt changes before a satisfactory solution is found.

Human-centric metrics like #attemptk (i.e., the average number of prompts used to instruct an LLM to obtain a satisfactory solution) and direct human interaction with LLMs offer insights into usability, understandability, and alignment with developers' interactions and needs, but these metrics are subjective, time consuming, and expensive. In addition, similarity metrics such as BLEU are often considered suboptimal for code generation, as they were originally designed for machine translation. BLEU primarily measures n-gram overlap between a candidate text and a reference text. While this works reasonably good for natural language, where semantic similarity often correlates with lexical overlap, code has a much stricter syntax and semantics. Hence, CodeBLEU metric attempts to address these limitations by incorporating program-specific features which extends traditional BLEU by including four sub-metrics: n-gram match (traditional BLEU), weighted n-gram match (assigning different weights to token types), Abstract Syntax Tree (AST) match (capturing syntactic similarity), and data flow match (evaluating semantic equivalence through data flow graphs). While described as more accurate and better adapted for code generation, CodeBLEU still has recognized limitations. It can be overly strict and underestimate a model's performance. It has been found to perform no better than more generic metrics from machine translation in correlation with human assessment. N-gram-based components of CodeBLEU still suffer from a poor correlation with human scores because of their inability to capture deep semantic meaning.

The best strategy to evaluate the code generated by LLMs is to have multiple metrics, including execution-based metrics, similarity metrics, and performance metrics. Additionally, developing dynamic evaluation systems to keep pace with the rapid evolution of LLMs is necessary to adopt an adaptive, holistic approach to evaluating code quality for real-world applications.

More details on the metrics used by researchers and their categories are shown in Table 2.2, and briefly discussed below:

- **Functional Correctness:** This category evaluates the generated code by checking if it is functional and produces the correct output. A common metric in this category is Pass@k, which measures how many of k codes passed a set of predefined test cases. This is mostly done automatically, which makes it popular among researchers as it is easy, fast, and cheap.
- **Syntactic Closeness/Similarity:** This category measures the similarity of generated

code to the reference code (human written code), where this includes syntax, variable names, code length, etc. The most commonly used metric for code similarity is BLEU.

- **Code Complexity:** This category measures how complex the generated code is to read, debug, or maintain. The most used metric among researchers in this category is LOC (Lines of Code), because it is easy and fast to measure the number of lines in a code. But it has some limitations, such as not taking into consideration code quality, structure, readability, and longer code is not necessarily more complex or hard to maintain. Moreover, LOC can also be affected by other factors like formatting styles or language syntax.
- **Code Performance:** This category measures the performance (e.g., execution speed and memory usage) of the code. The most adopted metric is Time Complexity metric, which measures how fast a code can be executed completely. This is easier to measure than Space Complexity (i.e., memory usage), especially in dynamic software where different inputs and runtime conditions are used.
- **Usability and Productivity:** This category measures how well an LLM assists developers with their coding tasks. The Generation Speed metric, which measures how fast code is generated, is the most used metric because faster output boosts productivity in software development.

Table 2.2: Evaluation metrics for LLM-generated code

#	Categories	Metrics	Published Papers
1	Functional Correctness	Pass@k	[20], [22], [23], [25], [59], [45], [56], [53], [37] ,[38] ,[39]
		Accuracy Rate	[4], [9], [60]
		Acc@K	[22]
		Pass@TopK	[61]
		Success Rate / Pass Ratio	[28], [35], [38], [40], [48], [61]
		Exact Match Accuracy	[47],[58]
		Computational Accuracy	[47], [27]
		Compilation Accuracy	[58]
		Compilation Rate, Match Success Rate, Code Extraction Success Rate	[27]
2	Syntactic Closeness/ Similarity	BLUE	[22], [43], [60], [33], [34] ,[53], [34],[58], [41]
		CodeBLEU	[8],[42],[43],[34],[38], [41]
		text2vec	[4]
		Jaccard Similarity	[22]
		SacreBLEU	[58]
		Normalized Levenshtein Similarity	[42], [35]
		OpenAI Text-Embedding Ada-002	[18]
		ROUGE	[33], [41]
		Perfect Predictions (PP)	[50]

		METEOR	[33], [41]
		RUBY	[41]
		ChrF	[41]
		Longest Common Subsequence (LCS)	[35]
		San Martino's Token Overlapping Metrics	[36]
		Abstract Syntax Trees (AST)	[61]
		SentenceBERT + Cosine Similarity (SBCS)	[34]
3	Code Complexity	Lines of Code	[17], [26], [42], [45], [52], [62]
		Cyclomatic Complexity	[17], [1], [45], [61]
		Token Count	[17], [18]
		Cognitive Complexity	[1], [45]
		Line Width	[26]
		Halstead complexity metrics	[16]
		McCabe cyclomatic complexity	[42]
		Words Count	[18]
4	Code Performance	Time Complexity	[17], [28], [42], [44], [59], [48]
		Space Complexity	[17], [28], [44], [48]
5	Usability and Productivity	Generation Speed	[9], [20], [8], [56]
		Average Completion Time	[56]
		Response Received	[61]
		#AttemptK	[56], [44]

2.2.7.3 Use Cases and Applications (RQ3)

According to the papers analyzed in this survey, a number of use cases have been identified on how developers use LLMs. The authors in [32] [63][54] used the DevGPT dataset, which was constructed from real conversations of developers with ChatGPT, to identify how developers were using it. In addition to that, the authors in [64][2][65] investigated with real developers by interviews and conducting surveys to assess how ChatGPT helped them with their software development tasks. Table 2.3 summarizes the most common use cases among developers, which are discussed as follows:

- **Code Generation and Refactoring**: Results show that developers often use ChatGPT to generate code from scratch and to refactor existing code. The generation process can be an entire function or code segment, while refactoring may involve improving performance or readability. For instance, developers rely on ChatGPT to write entire functions or code segments to integrate into their software projects. Furthermore, suggestions from ChatGPT were helpful for developers to write more efficient code [63], and they often used ChatGPT instead of web searching, as it provided faster solutions [2].
- **Learning, Explanation, and Educational Support**: Results show that developers obtain useful resources from LLMs, for instance, using ChatGPT, developers learn new programming libraries, including machine learning libraries (e.g., PyTorch and TensorFlow) and Application Programming Interfaces (APIs) [32]. Also, ChatGPT was used to explain different sections of code for developers in detail [54].
- **Code Optimization, Formatting, and Quality Assurance**: Studies have shown that

LLMs are used by developers for code optimization and formatting, specifically to organize code structures and follow best practices and coding standards. ChatGPT has assisted developers in restructuring code, correcting indentation, and using standardized naming conventions, as well as offering recommendations to improve code quality and readability [32].

- **Documentation and Deployment**: The studies have highlighted the use of LLMs in assisting developers in creating documentation for their projects. Studies have shown that developers used ChatGPT to write and edit documentation, such as README files and code comments [63].
- **Specialized Tasks**: LLMs are used with tasks beyond writing code by developers. Research has shown that ChatGPT has been used for specialized tasks such as networking and data manipulation. Additionally, ChatGPT has been used to address complex Software Development Kit (SDK) and API problems, such as the Amazon Web Services (AWS) SDK (Boto3) and the Nylas SDK [32].

Table 2.3: Summary of LLM Usage categories and actions.

#	Category	Action
1	Code Generation and Refactoring	Write this code for me. Improve the following code. Fix this issue in the following code. Solve the following problem/error in this code. Help to fix this error in my code.
2	Learning, Explanation, and Educational Support	Examples to use the following APIs. Explain this code for me. Request more description about the following code/function.
3	Code Optimization, Formatting, and Quality Assurance	Request improvements for this code (optimization). Add instructions. Request verification. Find mistake/error then request to fix it. Test this code with this input.
4	Documentation and Deployment	Add more context/comments/description. Request examples on how to use this program/function.
5	Specialized Tasks	Request networking, bioinformatics, or APIs.

2.2.7.4 Prompt Design and Effectiveness (RQ4)

The analysis of the selected studies shows that several researchers have studied the effect of prompts on code generation tasks. In [9], the authors stressed the role of prompts in generating code, highlighting that prompts focused on security improve the security of generated code and prompts that explicitly address security issues significantly reduce vulnerabilities. Similarly,

the authors in [22] studied the impact of different prompts on the quality of generated code, showing that semantically clear prompts have improved ChatGPT's performance. In their study [43], the authors showed that a well-crafted prompt improved the code generation process after evaluating three different prompts to guide ChatGPT in Text-to-Code (T2C) tasks. In another study [66], the authors used Chain-of-Thought (CoT) prompts and compared them with traditional prompting techniques, suggesting that prompts focused on optimization enhanced the efficacy of generated code, especially for complex tasks. Additionally, the authors in [39] proposed a novel prompting strategy called Structured CoT (SCoT), which improved the performance of LLMs in code generation tasks.

However, even a well-engineered prompt can fail under certain circumstances, such as when the task description is insufficient or when details are missing, LLMs can generate code that deviates from the user's needs. Additionally, long prompts and multi-round interaction with these models can exceed a model's token limit, leading to generate incomplete or incorrect codes. Given the non-deterministic behavior of LLMs, where "the same prompt generates different solutions on different attempts", it is a challenge for researchers to determine whether the proposed output is the best, particularly with complex and CoT prompts. An overview of prompt strategies is shown in Table 2.4. The reviewed studies show that LLMs' success in code-generation tasks strongly depends on prompt design; with well-structured prompts, LLMs can enhance accuracy, security, and the optimization of generated code. This implies that the effective implementation of prompt engineering is crucial for optimizing LLM use.

Table 2.4: Common prompts strategies.

#		Strategy
1	Core Prompting Strategies	Zero-Shot Prompting [35] Few-Shot Prompting [62] Chain-of-Thought (CoT) Prompting [38] Structured CoT (SCoT) Prompting [39] Template-Based Prompting [43]
2	Contextual and Framing Strategies	Context-Rich Prompting [64] Role-Based Prompting [9] Iterative/Multi-Round Prompting [17]
3	Control and Constraint-Based Strategies	Conciseness Requests Prompting [43] Security-Focused Prompting [9] Efficiency Prompts [59] Format Control Prompting [27]

2.2.7.5 Security of LLM-Generated Code (RQ5)

In many studies including [1], [19], [21], [29],[67], [31],[32], and [58], the security of generated code has been evaluated to find possible vulnerabilities in these codes, the researchers found many vulnerabilities such as bad resource management, insecure coding patterns, and hardcoded credentials in codes generated by various LLMs.

The authors in [1] evaluated the generated code from several LLMs, including ChatGPT, Claude, Spark, and Bing AI. Their results show that these models generated vulnerable codes. Similarly, the authors in [21] explored the vulnerabilities in code generated by different LLMs, such as ChatGPT, Copilot, and CodeGen, and concluded that these models can generate code with exploitable vulnerabilities (e.g., unauthorized access). In another study [19], GitHub Copilot was evaluated, where researchers used CodeQL and manual inspection to find vulnerabilities in the generated code. Their experimental results indicated that the code still contains a range of vulnerabilities even after Copilot's internal security filters (post-processing).

Other studies focused on evaluating ChatGPT in real-world development environments. The authors in [32] studied real developers' interaction with ChatGPT for generating code, they found many security issues (e.g., insecure comments) and other code quality issues (e.g., undefined variables) that these codes needed extensive modification before adding to projects, concluding that ChatGPT's output has poor quality and cannot be added to repositories or used without modifications. In another study by the authors in [67], the code generated by ChatGPT was compared with codes from the Stack Overflow platform; they found that both have insecure codes, but ChatGPT tends to generate code with less vulnerabilities compared to codes in Stack Overflow.

In summary, many researchers have found that the generated code is insecure, even though modern LLMs show better performance, such as ChatGPT's code, which poses fewer security risks than community-based platforms like Stack Overflow. LLMs cannot be trusted to generate secure code, therefore, developers should treat this code as insecure by default, and security tools such as CodeQL and extensive manual inspection and modification should be applied before adding them to any software project.

2.2.7.6 Benchmark/Dataset Characteristics (RQ6)

Benchmarks are constructed by researchers to evaluate the performance of LLMs for code generation and understanding. These benchmarks usually consist of code segments, functions, classes, full programs, or algorithms. They also include prompts to feed to LLMs, whether in natural language or partial code, to guide the code generation process. Additionally, test cases are included to verify that the generated code is syntactically and semantically correct. Reference solutions are also usually incorporated as ground truth to verify the reliability of validation and comparison with the generated code. The vast majority of datasets are constructed from public repositories such as databases, coding websites, and textbooks, with

links cited to maintain reproducibility and transparency. After an extensive review of the literature, several benchmarks were used by researchers. The identified benchmarks including the dataset name, type of programming language used, availability of test cases, reference solutions, data sources, and dataset link are reported in Table 2.5. According to the survey results, HumanEval is the most commonly used benchmark in the previous studies, as it provides common well-defined Python programming problems and corresponding test cases. Its structured design also allows consistent comparisons across models, which helps in reproducibility and comparability of research. However, recent studies do not use HumanEval to evaluate newer LLMs, as it may have been included in the training data of LLMs, potentially introducing biases into the results.

Table 2.5: Benchmarks used for evaluating code generation in LLMs.

#	Name	Code Type	Programming Language	Source of Data	Test Cases	Reference Solutions	Link
1	LLMSecEval [31]	Code segments	Python and C	MITRE's Top 25 CWE, Authors	-	Yes	https://github.com/tuhh-softsec/LLMSecEval/
2	DevGPT [16], [63], [54][68]	Functions, classes, algorithms, and full programs	Multiple languages	Chats between developers and ChatGPT	-	-	https://github.com/NAIST-SE/DevGPT
3	OJI Dataset [53]	Full programs, functions, and algorithms.	C++, Python Code	Romanian Informatics Olympiad (2002 to 2023)	Yes	Yes	-
4	HumanEval [20], [23], [59] [35], [37], [39]	Functions	Python	Authors	Yes	Yes	https://github.com/openai/human-eval
5	Human-Eval – ET [20]	Functions	Python	Human-Eval with additional test cases	Yes	Yes	
6	MBPP [20], [59], [37], [39] [59]	Functions, classes, programs	Python	Coding platforms and competitive programming problems.	Yes	Yes	http://github.com/google-research/google-research/tree/master/mbpp/
7	MBPP-ET [20], [37]	Functions, classes, programs	Python	MBPP with additional test cases	Yes	Yes	-
8	APPS [4], [35], [37]	Functions, algorithms, and complete programs	Python	Programming platforms and online coding competitions	Yes	Yes	https://github.com/hendrycks/apps
9	LLMC Dataset [1]	Functions, classes, programs	Python	LeetCode, Authors	Yes	Yes	-
10	MBPP+ [23]	Functions, classes, programs	Python	Extended version of MBPP	Yes	Yes	https://github.com/evalplus/mbppplus_release
11	Bash dataset [23]	Bash scripts	Bash scripts	Authors	Yes	Yes	-
12	CodeLMSec [21]	Functions, classes, and algorithms	Python and C	Generated using GPT-4 and Code Llama-34B	-	-	https://github.com/codelmsec/codelmsec
13	CoderEval [22], [37]	Functions and non-standalone functions	Python and Java	Open-source projects	Yes	Yes	https://github.com/CoderEval/CoderEval

14	ClassEval [25]	Classes with multiple methods	Python	Authors	Yes	Yes	https://github.com/FudanSELab/ClassEval
15	CodeXGLUE [60]	Functions, classes, and programs	Java, C#, Python, Ruby, Go, C/C++, JavaScript, PHP	Open-source repositories and coding platforms	Yes	Yes	https://github.com/microsoft/CodeXGLUE
16	R Dataset [56]	Functions and Algorithms	R	R programming textbooks	Yes	Yes	-
17	CodeNet [27]	Functions, algorithms, and programs	C, C++, Go, Java, and Python	Open-source projects and coding platforms	Yes	Yes	http://github.com/IBM/Project_CodeNet/
18	T2C Dataset [43]	Functions	Java	Part of the CodeXGlue benchmark	-	Yes	https://github.com/BaoBaoGitHub/guiding-chatgpt-for-code-generation
19	C2C Dataset [43]	Functions	C# and Java	Part of the CodeXGlue benchmark	-	Yes	https://github.com/BaoBaoGitHub/guiding-chatgpt-for-code-generation
20	ManySStuBs4J [66]	Single-statements	Java	Open-source projects	-	Yes	-
21	LeetCodeEval [59]	Functions	C++	LeetCode Problems	Yes	Yes	https://github.com/NougatCA/EfficiencyEval
22	ScenEval [45]	Functions, algorithms, and programs	Java	Textbooks, W3Resource, and Stack Overflow	Yes	Yes	-
23	HDLEval [52]	Code relevant to hardware design	Verilog, Chisel, pyRTL, and DSLX.	HDLBits	-	Yes	-
24	VHDL-Eval [55]	Digital logic design and hardware description tasks	VHDL	Verilog-Eval and VHDL tutorials	Yes	Yes	-
25	VerilogEval [53]	Hardware design tasks	Verilog	HDLBits	Yes	Yes	https://github.com/NVlabs/verilog-eval
26	CopilotEvaluation [61]	Sorting algorithms, Data structures	Python	Programming courses and books	Yes	Yes	http://github.com/Copilot-Eval-Replication-Package/CopilotEvaluation/
27	Shellcode_IA32 dataset [58]	Assembly code segments	Assembly Language (IA-32)	Publicly Available Security Exploits:	-	Yes	https://github.com/dessertlab/Shellcode_IA32
28	IEEEXtreme [40]	Functions, algorithms, and programs	Python 3, Java 7, and C++	IEEEXtreme Competition	Yes	Yes	http://www.kaggle.com/datasets/riotulab/chatgpt-evaluation-on-ieeextreme-competitions/

2.2.7.7 Models used in Evaluations (RQ7)

After analyzing all the relevant studies, it becomes clear which LLMs were used in each study. Models like ChatGPT (versions 3-5) and other specialized variants appear frequently in different studies, with some research evaluating more than one LLM. Table 2.6 highlights the LLMs used in code generation research. When selecting models for evaluation, researchers balance two axes of choice: licensing (open-source vs. proprietary) and architecture (decoder-only vs. encoder-decoder). Proprietary models, such as OpenAI's GPT-3.5, GPT-4, Anthropic's Claude, and Google's Gemini, dominate many applied studies due to their high performance and convenient APIs access. GPT-4, for example, regularly tops benchmarks for functional correctness and security and generates code with extensive comments [26]. However, the lack of transparency around the training data used and the internal functionality limits researchers' ability to evaluate them. By contrast, open-source models such as

CodeLlama, StarCoder, PolyCoder, Mistral, and CodeGen are fully inspectable and fine-tunable. Recent results show that CodeLlama-Python-7B can outperform much larger closed-source models on benchmarks such as HumanEval [59], and Mistral variants excel at automated vulnerability repair [50]. Yet, smaller open-source models struggle to generate correct solutions from start to finish and are often constrained by their maximum context size.

Most studies focus on decoder-only transformers, which are designed to predict the next token based on a given input context. They generate code autoregressively, token by token, based on the preceding context and generated tokens [3]. The GPT-series (GPT-3/3.5/4), Codex, and open-source models such as CodeLlama, StarCoder, and DeepSeek Coder fall into this category. These models excel at generative tasks where the output flows sequentially from a given prompt (like code completion or initial code generation from a natural language description). In contrast, encoder-decoder models (e.g., BART-derived CodeT5, PLBART, AlphaCode) comprise two main parts: an encoder and a decoder. The encoder processes the input (e.g., natural language description), converting it into a fixed-length representation (i.e., vector), and the decoder then generates the output (e.g., code) based on this representation [43]. Such a bidirectional encoding step can enhance tasks that require a comprehensive understanding of the input, such as code summarization or language translation, but it is not as widely used for generating-from-scratch tasks.

Table 2.6: LLMs used for code generation evaluation.

Model	Published Papers	Total
ChatGPT – 3.5	[4], [16], [9], [23], [24], [25], [8], [42], [44], [45], [69], [52], [46], [64], [58]	14
ChatGPT 4	[57], [20], [25], [8], [26], [28], [59], [69], [52], [56],[34] , [46], [53], [35], [36], [37], [48]	15
ChatGPT (Version not mentioned)	[17], [1], [63], [56], [30], [45], [54], [34]	9
ChatGPT – 3.5 Turbo	[18], [22], [43], [29], [67], [3], [59], [52], [53], [35], [47], [34], [37], [39]	12
CodeLlama-Python-7B	[20], [26], [26], [27],[59], [55], [69], [47], [34], [37]	10
Codellama 34B	[23], [26], [27], [59], [55], [37]	6
Codellama-instruct-13b	[49], [27], [59], [69], [47]	4
Starcoder	[23], [25], [26]	3
WizardCoder 15B	[25], [27], [59]	3
Instruct-CodeGen 16B	[25], [67], [52], [37], [38]	5
CodeGen (350M)	[22], [67], [52]	3
Gemini 1.0	[9], [8], [26]	3
mistral-7b-instruct-2	[49], [26]	2
mixtral-8x7b-instruct	[49], [27]	2
InCoder-6B	[20], [25], [37], [38]	4
InCoder-1.3B	[36]	
CodeParrot-1.5B	[36]	
CodeGeeX2-6B	[20], [67]	2
CodeGen2.5-7B	[20], [55], [36]	2
GitHub Copilot	[19], [42], [61]	2
PolyCoder 2.7B	[25], [67], [36], [34]	4
Codex	[67], [31], [37], [41]	2
ChatGPT – 3.4	[16], [9]	2

DeepSeek-6.7B	[20], [26], [39]	3
MagiCoder-6.7B	[20], [27]	2
Microsoft's Bing AI	[1]	1
iFLYTEC's Spark	[1]	1
Anthropic's Claude	[1]	1
CodeT5+-Python-770M	[20], [60], [34], [58]	4
DeepSeek Coder (33B Base, 33B Instruct)	[59]	1
ChatDev	[20]	1
PanGu-Coder (300M)	[22]	1
Codellama	[51], [50]	2
Mistral 7B	[23], [50]	1
RoMistral 7B	[26]	2
ChatGLM 6B	[25]	1
Vicuna 7B	[25]	1
CodeGeeX 13B	[25], [37], [38]	3
Instruct-StarCoder 15B	[25], [46], [34], [37]	4
Copilot	[17]	1
Codestral 22B	[26]	1
AutoCoder 6.7B	[26]	1
CodeQwen 1.5 7B	[26]	1
Yi 9B	[26]	1
Phi3 14B	[26]	1
Tabnine	[42]	1
Google Bard	[42]	1
Phi-2	[59]	1
CodeGen2.5-QLoRa	[55]	1
Granite-20B-Code-Instruct	[55]	1
Granite-20B-Code-Instruct-ICL	[55]	1
VeriGen (6B, 16B)	[69]	1
TransCoder	[47]	1
AlphaCode (1.1B)	[37][38]	2
CodeBERT	[58]	1

2.2.7.8 Code Analysis Tools used in Evaluations (RQ8)

To evaluate the quality of code generated by LLMs, researchers have to do it manually, which can be time-consuming and expensive. Therefore, many researchers have used available static code analysis tools such as PyLint, Flake8, or SonarQube to automate this process, allowing them to check the security, style, and complexity of generated code. Furthermore, dynamic evaluation tools such as LeetCode Online Judgement are used by researchers to assess the functionality and performance of the generated code using the available test cases. CodeQL is used across many studies because it is a powerful tool for vulnerability detection across many programming languages, and its integration with a variety of analysis scenarios makes it a reliable tool among researchers for code quality and security. The purpose and programming language these tools support is shown in Table 2.7.

However, such tools have limitations. While PyLint is generally very precise but still generates false positives and often focuses only on style issues (e.g., newlines and whitespaces)

and import messages (e.g., unused or missing imports) [32]. Flake8 prioritizes quantifiable style violations such as whitespace and syntax over qualitative aspects, such as code readability [4]. SonarQube, despite supporting multiple programming languages, has cognitive complexity metrics limited to Java, Python, and JavaScript, and it has few relevant queries for detecting vulnerabilities in Python or JavaScript [3]. CodeQL, while powerful for identifying security vulnerabilities and checking code quality, it has vulnerability capabilities that are limited to specific types, such as pointer and memory-related vulnerabilities in algorithm problems for languages like C, C++, and Java, and may not cover other languages like Python [3]. The LeetCode Online Judgement platform assesses runtime and memory utilization. Still, it terminates execution upon the first test failure, meaning the test case pass rates provided may serve as a lower bound rather than a complete assessment of potential correctness beyond the first failure [3].

Table 2.7: Code analysis tools used for evaluating LLM-generated code.

#	Tool	Purpose	Supported Languages	Official Link
1	CodeQL [19], [21], [67], [3], [32]	Code vulnerability analysis	C/C++, C#, Java, JavaScript, Python, Ruby, Swift and more.	https://codeql.github.com/
2	Flake8 [8], [30]	Style guide enforcement	Python	https://flake8.pycqa.org/
3	PyLint [32]	Static code analysis	Python	https://pylint.pycqa.org/
4	Bandit [32]	Security analysis	Python	https://bandit.readthedocs.io/
5	PMD [54]	Code quality and style checker	Java, Apex, JavaScript, XML, XSL and more.	https://pmd.github.io/
6	CheckStyle [44]	Style and convention checker	Java	https://checkstyle.org/
7	LeetCode Online Judgment [3], [48]	Functional correctness evaluation	C++, Java, Python, JavaScript, Go and more.	https://leetcode.com/
8	SonarQube [1]	Code quality and security	30+ languages including Java, C#, Python, JavaScript, TypeScript, C/C++ and more	https://www.sonarqube.org

2.2.7.9 Evaluation Challenges (RQ9)

Various challenges were identified when using LLMs for code generation, especially in evaluating their ability to generate accurate and efficient code. The challenges are listed below:

- **Lack of Evaluation Metrics:** There are still no universally accepted metrics for assessing the code generated by LLMs. Also, metrics like Pass@k and BLEU are used in many evaluation studies, but they may not reflect the full quality, correctness, or security of the code [70].
- **Functional Correctness and Syntactic Similarity:** A particular issue is the balance between functional correctness (does the code do what it was intended to do?) and syntactic similarity (how closely does the generated code resemble the reference solution?). Functional correctness is important, but it is difficult to be automatically

evaluated, especially when dealing with complex tasks with no test cases. Similarity metrics, such as BLEU, may not accurately reflect the usability or correctness of generated code, leading to inconsistent findings [45].

- **Security:** The code generated by LLMs can contain security vulnerabilities within Common Weakness Enumeration (CWE), and assessing the security of generated code requires specialized tools such as CodeQL, but even these tools can lead to false positives or fail to identify certain vulnerabilities [19].
- **Complexity and Maintainability:** The generated code can be functionally correct but complex and difficult to maintain. Complexity metrics such as CC and LOC can evaluate code complexity, but they still do not provide full insight on the readability or maintainability. These features often require human judgment to evaluate, which is subjective and time-consuming [16].
- **Benchmark Limitations:** Most of the benchmarks that were used to evaluate LLMs are either constructed from a single source or do not consist of diverse programs, which can potentially lead to biases in the results. However, datasets like HumanEval and MBPP are commonly used but do not cover real-world programming tasks or applications. Consequently, the evaluation results may not be reliable [70].
- **Prompt Sensitivity:** The quality of generated code strongly depends on the prompt design. Even small changes to the prompt can generate fundamentally different outputs, which makes it difficult to construct a prompt that generates quality code. Because of prompt sensitivity, the evaluation process is complicated, as results may vary more in response to prompt design than the model's capabilities [43].
- **Token Limitations and Incomplete Code:** LLMs can generate incomplete or cut-off code segments due to their token limitations. This issue can particularly occur with complex, large code tasks, as the code may be incomplete and fail to produce the expected output, making it difficult to determine whether it is incorrect or incomplete [3].
- **Cross-Language Evaluation:** LLMs are typically evaluated in only a single programming language (e.g., Python), though their performance may differ for other languages and application domains. Assessing the performance of LLMs for cross-language code generation (e.g., translating code from Python to Java) presents more challenges due to syntax and best practices across programming languages [24].
- **Evaluation of Multi-Round Fixing:** LLMs may require multiple prompts and attempts to achieve the desired output. Evaluating the effectiveness of these multi-

round prompts is challenging because it requires tracking the development of the output code across several iterations and the final code may be affected by the quality of the user's feedback and prompts [3].

- **Lack of Hardware Code Evaluation:** Most LLMs are evaluated mainly on high-level programming languages (e.g., Python and Java); very few are evaluated on Hardware Description Languages (HDLs) such as Verilog and VHDL. This limits their usefulness in hardware design and verification, where domain knowledge and syntax are required [51].
- **Efficiency and Cost**: LLMs still require expensive, high-performance GPUs for training and efficient operations [62]. This leads many researchers to use cloud-based models or through paid APIs, which are costly.

2.2.7.10 Future Research Directions (RQ10)

Several possible directions for future research were identified based on the analysis of the papers in literature. These future directions are summarized and categorized under the following points:

- **Evaluation of Non-Determinism:** Future research should address the non-deterministic nature (i.e., the inconsistency in the generated code candidates across different requests with identical prompts) of LLMs by developing evaluation frameworks that account for variability in generated outputs. This challenging task requires finding new methods to decrease randomness and improve consistency as well as to studying the impact of prompts on non-determinism.
- **Development of new Metrics:** Future research can involve developing metrics that evaluate not just whether a program works correctly (functional correctness), but also its quality (e.g., readability), security (e.g., access control, data validation), performance (e.g., execution time, memory usage), and maintainability (e.g., ease of modification). In addition, the developed metrics need to be reliable and correlated with the judgments made by humans in order to provide accurate evaluation results.
- **Evaluation of Code Summarization and Documentation:** Future research needs to analyze LLMs' capacity to generate code summaries, comments, and documentation. This means assessing how readable, relevant, and complete the generated explanations are.

- **Development of Benchmarks:** Future work should focus on creating benchmarks for real-world coding tasks, including complex dependencies, the use of external libraries, and even including complete projects.
- **Evaluation of Resource-Constrained Devices:** Future work should focus on the ability of LLMs to generate code for resource-constrained IoT devices, such as Arduino and Raspberry Pi. This includes evaluating the efficiency of the code that should be evaluated in terms of execution speed, Flash memory, and Static Random Access Memory (SRAM) usage.
- **Multilingual Evaluation:** Future research can compare the ability of LLMs on generating using prompts in other languages (e.g., Kurdish or Arabic).
- **Security and Correctness:** Additional research could examine the effects of different prompting on security and correctness of the generated code. Then, to assess the tradeoff between code security and functionality.
- **Quality and Consistency:** The quality and consistency of the generated code regarding several different metrics and a variety of programming languages should be studied further. Studies should also be conducted regarding the practical feasibility of the generated code for use in actual projects and how many changes/modifications the code will have to undergo before it can be used as part of a project.
- **Evaluation of New LLMs**: Newer LLMs can be evaluated for their performance in tasks, such as hardware design (e.g., Verilog, VHDL), low-level programming (e.g., Assembly), and IoT resource-constrained devices (e.g., Arduino).
- **Prompt Engineering Techniques:** Future studies should explore how to manually construct and how to automate the construction of prompts for code generation. In addition, future research should be conducted into developing frameworks that automatically create, refine, and improve prompts using a variety of types of feedback (e.g., successful execution, style metrics, etc.)

2.3 Comparison with Related Studies

This section briefly presents the most relevant publications to this thesis, and are summarized in Table 2.8. The authors in [57] have explored how to integrate ChatGPT into embedded systems using the Arduino platform through the creation of two case studies. In the first case study, the authors demonstrated how ChatGPT can be used to classify sensor data captured from the Arduino using classification methods. In the second case study, the authors demonstrated how ChatGPT can be utilized to generate template code for common use cases

that developers would typically implement manually when developing applications for the Arduino platform. While the study demonstrated how ChatGPT can potentially improve the rate at which developers can develop software, the authors did note limitations, including issues related to retaining the context of the problem as well as issues with the accuracy of generated code. However, the study only used one LLM (ChatGPT) and only two use cases, which lacked the systematic, multi-model, and multi-metric analysis that are necessary to fully understand the potential of LLMs for improving the development process for embedded systems.

The authors in [1] developed an evaluation framework to evaluate LLMs' ability to generate code across six evaluation aspects (validity, correctness, complexity, reliability, security, and readability). Additionally, they developed the LLMC dataset, which includes 45 Python coding problems to evaluate four different LLMs. Similar to this is the work presented by the authors in [48], who performed an empirical analysis of ChatGPT's ability to solve common programming problems in C++ and Java. The authors generated a large data set that included examples from many different categories and difficulty levels and then used a structured experimental design to measure the correctness, execution speed, and memory use of the generated code compared to the code by human programmers. The authors in [56] analyzed the effectiveness of ChatGPT as an R code generation tool using a user-centered research methodology. They measured both code quality (i.e., accuracy, readability, and conciseness) and user experience. ChatGPT was able to generate high-quality, accurate, and readable code with good explanations but scored low on conciseness. Most recently, the authors in [71] compared three LLMs (GPT-3, Gemini, LLaMA, and Claude) for generating Python code. The authors tested each model on ten programming tasks that varied in their level of complexity and used the following evaluation criteria: syntax correctness, accuracy, reliability over repeated attempts, response time, cost, and error handling.

While these studies have useful approaches for determining whether LLM-generated code is reliable and efficient using general programming languages (Python, Java, etc.), they cannot be translated directly into the unique constraints of resource-constrained IoT devices (e.g., Arduino) where memory utilization and processing time are important. In [72], the authors studied the combination of IoT and LLMs and found that there were several primary applications of IoT with LLMs: smart home, healthcare, transportation, manufacturing, and environment. These authors showed how natural language interfaces added value to IoT systems. Although the authors presented an overview of some of the challenges related to edge computing (resource limitations) without providing an empirical evaluation of code generation for resource-constrained IoT devices. The authors in[69] developed CreativEval, a method for evaluating the creativity of LLM-generated High-Level Description Language (HDL) code

based on both novelty and functionality. In their research, they evaluated solutions to various hardware design tasks and compared responses from different LLMs, specifically ChatGPT and Code Llama. The authors in [45]] introduced ScenEval, a benchmark designed for assessing the generated code through scenario-based evaluations, they used ChatGPT to evaluate with a variety of metrics, including functional correctness, cognitive complexity, and the average number of iterations to generate correct code. Although they used ChatGPT to assess the generated code using the aforementioned metric, their evaluation is limited to a single LLM and a general-purpose language (Java). Therefore, there is no systematic and multi-model comparison of performance, nor is there an investigation of other important performance metrics (such as execution time). In addition, the authors in [73] have recently completed an empirical study of ChatGPT's performance using several datasets to assess its ability to generate code. Through their analysis, they demonstrated that ChatGPT has a strong performance in generating code for brief and concise problem descriptions. However, they demonstrated that when given narrative problem descriptions, ChatGPT had difficulty identifying the problem and developing a plan of action to solve it. They then examined additional programming languages (Python and C++), iterative prompting, and targeted feedback loops as methods to improve the accuracy and efficiency of ChatGPT. Despite these contributions, their work again remains limited to a single LLM (i.e., ChatGPT-4) and focused on accuracy and runtime performance.

Broader aspects such as memory consumption and systematic multi-model comparisons remain unaddressed. Beyond code generation, recent research has explored enhancing LLMs for specialized tasks through improved prompting. For instance, the authors in [43], conducted an empirical study on guiding ChatGPT for improved code generation using prompt engineering techniques. They evaluated ChatGPT’s performance on two tasks: text-to-code and code-to-code generation, using the widely adopted CodeXGlue benchmark. They used CoT prompting and multi-step optimizations, exploring factors such as prompt specificity, conciseness, session settings, and generation randomness. Similarly, the authors in [74] used meta-reasoning prompting to boost the efficiency and task-adaptive reasoning of models like LLaMA in multi-modal contexts. Their work underscores the importance of sophisticated prompt design and model fine-tuning for specialized domains, a consideration that aligns with the challenges of generating efficient code for resource-constrained environments.

While previous studies have investigated LLMs for generating code for general-purpose programming languages and some have explored basic integrations and usage of LLMs with Arduino, none have systematically evaluated LLMs for Arduino code generation. Unlike prior research, which focused on single LLMs, limited task types, or a narrow set of metrics, this

work provides a comparative analysis of six state-of-the-art LLMs across 31 performance-focused Arduino coding tasks. Beyond functional correctness, also measuring execution time, memory usage, code complexity, and code similarity to reference code were employed. The findings reveal each LLM's trade-offs between speed, efficiency, and maintainability, contributing to improvements in the reliability and efficiency of LLMs for code generation for resource-constrained devices.

Table 2.8: Summary of comparsion with related studies.

Paper	Target Language	Evaluated LLMs	Used Datasets	Prompting Types	Used Metrics/Methods
[74]	N/A	LLaMA-7B with custom adapters + MRP layers	GSM8K, MMLU, ScienceQA, Alpaca-52K, HotpotQA, LVLM-eHub	Meta-Reasoning Prompting (MRP).	Accuracy, Params, Training time, Strategy Switch Rate, BLEU@4.
[72]	N/A	GPT, LLaMA, Claude, Gemini, Mistral, BERT (reviewed).	N/A (Literature Review)	Survey of Prompting Strategies.	Accuracy, latency, memory, power (reviewed).
[1]	Python	ChatGPT, Claude, Spark, Bing AI	LLMC Dataset (45 tasks).	Manual Standard / Template-Based Prompting.	Validity, Correctness (pass rate), Complexity, Reliability/Security, Readability.
[73]	C++, Python	ChatGPT (GPT-4).	LeetCode (102 tasks), Codeforces (150 tasks)	Few-Shot + Iterative / Multi-Round Prompting.	Accepted, Wrong Answer, Time Limit Exceeded, Runtime Error, Memory Limit Exceeded, Compile Error.
[71]	Python	GPT series, Gemini series, LLaMA 3, Claude 3 series	10 custom coding tasks of varying complexity.	Standard / Template-based Prompting.	Syntax, Completeness, Response Time, Accuracy, Reliability, Exception-handling, Cost, Efficiency.
[48]	C++, Java	ChatGPT (GPT-4).	LeetCode (240 tasks)	Efficiency / Conciseness Prompts.	Correctness, Runtime Efficiency, Memory Usage.
[43]	Java, C#	ChatGPT (GPT-3.5-Turbo).	CodeXGlue	Structured CoT + Template-Based Prompting	BLEU, CodeBLEU.
[57]	Arduino	ChatGPT (GPT-4).	Two Arduino case studies.	In-Context / Zero-Shot Prompting.	Prediction time, Context size, Accuracy, Code generation time, Compile errors.
[45]	Java	ChatGPT.	ScenEval dataset	Zero-Shot Prompting.	Pass@1, Avg Pass, Complexity (cyclomatic, cognitive, LOC).
[56]	R	ChatGPT.	R programming tasks (351).	Iterative / Multi-Round User-Centric Prompting.	Usability attributes (Acc, Completeness, Conciseness, Readability, etc.), Attempts, Completion time.
[69]	Verilog	GPT-3.5, GPT-4, CodeLlama, VeriGen	HDLBits	Structured / Creativity-Oriented Prompting	Fluency, Flexibility, Originality, Elaboration, Functionality, GNN4IP similarity.

This Thesis	Arduino	ChatGPT-4o, Gemini 2.0 Flash, DeepSeek-V3, Claude 3.5 Sonnet, GitHub Copilot, LLaMA-3	31 performance focused Arduino programs.	Zero-Shot Prompting	Syntactic and Functional correctness, Execution time, SRAM, Flash Memory, Multi-round Error Correction, Cyclomatic Complexity, NCLOC, CodeBLEU.

Chapter 3:

Methodology, Dataset, and Model

Chapter 3: Research Methodology

3.1 Introduction

This chapter presents the research methodology used to evaluate different LLMs in generating Arduino code. It introduces the five research questions of this thesis, describes the used dataset, and presents the six selected LLMs for evaluation. Also, it presents the used evaluation metrics for measuring correctness, performance, multi-round correction, complexity, similarity, and explains how each metric aligns with the research questions.

3.2 Research Methodology

The research methodology employed in this thesis consists of seven steps: identifying research questions, preparing the subject programs, selecting LLMs, defining evaluation metrics, prompting LLMs to generate codes, testing codes, and analyzing the results to evaluate LLMs' capabilities in generating correct, efficient, and high-quality Arduino codes, with each step described in detail in the following sections.

Figure 3.1 shows the used evaluation procedure. The experiments begin by retrieving a prompt (i.e., code question) from the used dataset and passing it to each LLM to generate an initial code solution. Then, the solution, using the Arduino hardware and the Arduino IDE software with its default settings, is evaluated both syntactically and functionally for overall correctness. If it fails the correctness check, the model is re-prompted with feedback (compiler feedback in the syntactic case or author feedback in the functional case). If the model continues to generate incorrect solutions, the process is repeated up to five times. If a solution successfully passes the overall correctness check, then its performance and complexity are analyzed. Otherwise, it is only considered in terms of code similarity and multi-round error correction. The limit of five rounds was selected because it provides a fair representation of a developer taking a reasonable number of attempts to repair a piece of code. As noted in [3], multi-round fixing typically sets a maximum of five rounds, since extending the limit to ten rounds resulted in only marginal improvements.

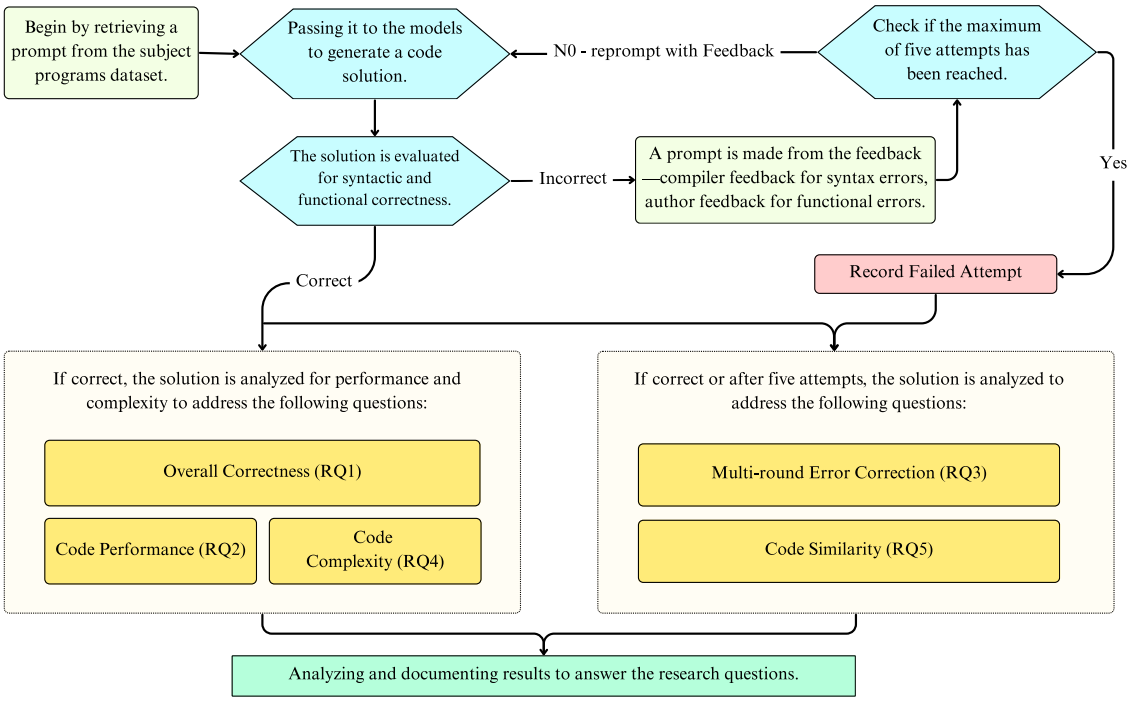

Figure 3.1: The procedure used to evaluate LLMs.

3.3 Research Questions

Several RQs have been identified and addressed in this thesis, each focusing on a specific aspect of the topic as follows.

- **RQ1 (Overall Correctness):** Is the code generated by LLMs syntactically and functionally correct?
- **RQ2 (Code Performance):** How efficient is the generated code in terms of runtime and memory usage?
- **RQ3 (Multi-round Error Correction):** Can iterative prompting improve the correctness of initially incorrect code?
- **RQ4 (Code Complexity):** Does the generated code reflect the same level of complexity as code written by human developers?
- **RQ5 (Code Similarity):** How similar is the generated code to the original developer-implemented code?

3.4 Used Dataset

The used dataset includes 31 optimized Arduino programs that were constructed using coding examples from the official Arduino reference website[1]. The subject programs cover a wide range of Arduino fundamentals, instructions, and tasks. For each program task, two reference solutions were prepared to reflect a developer coding variation and to ensure the best possible approaches were explored, as it was not always clear which coding strategy would perform better. Some programs were more efficient in terms of memory usage, while others offered lower execution times. The reference solutions were prepared with a strong emphasis on performance, and manual optimization techniques, to eliminate redundant computations, select appropriate data types, and optimize control flow, were applied to ensure efficiency, reliability, and high code quality. The used dataset and other materials are publicly available on GitHub[2].

3.5 Used LLMs

In this thesis, six well-known LLMs were selected for evaluation: ChatGPT-4o[3] by OpenAI, Gemini 2.0 Flash[4] by Google, DeepSeek-V3[5] by DeepSeek, Claude 3.5 Sonnet[6] by Anthropic, GitHub Copilot[7] (powered by GPT-4o) by OpenAI in collaboration with GitHub, and LLaMA-3[8] by Meta. Each one of these models is developed and maintained by a leading tech company, ensuring a diverse representation of current advancements in LLMs, and all were used with their default configuration settings.

3.6 Used Prompt Type

In this thesis, one popular prompting type was used to assess the performance of selected LLMs, which is zero-shot prompt [39]. It asks a model to do a task without offering any examples. Additionally, it clearly defines the role and expertise level, specifies the board, states the task (i.e., code question), and asks for optimized code for best performance. Notably, zero-shot prompting mimics common LLMs-users' interactions, as it asks for something directly when the model simply needs to work with its own internal knowledge, not an example from

[1] https://www.arduino.cc/reference/en/
[2] https://github.com/LLMsRes-ch/arduino-subject-programs/
[3] https://chatgpt.com/
[4] https://gemini.google.com/
[5] https://www.deepseek.com/
[6] https://claude.ai/
[7] https://github.com/copilot
[8] https://www.meta.ai/

a given task where users do not possess the required expertise or time. The used prompt is structured as follows:

> *"You are an experienced software developer. Write an optimized Arduino Uno Rev3 code that [code question]. Only write the code and ensure it's optimized for best performance."*

An example prompt and the code generated by ChatGPT are shown in Figure 3.2.

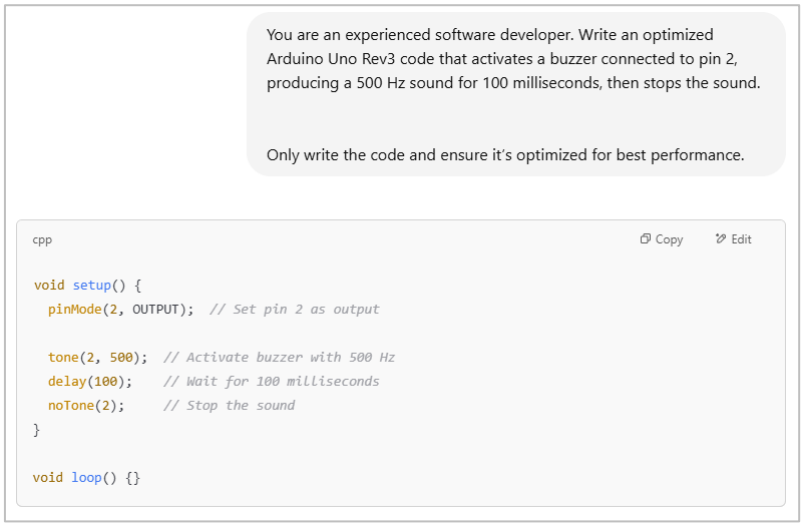

Figure 3.2: Example of prompt and code solution generated by ChatGPT.

3.7 Used Evaluation Metrics

To ensure the accuracy and reliability of the obtained results, a set of well-known evaluation metrics were used in this thesis, including CC [75] and CodeBLEU [76]. Additionally, two new metrics were employed for the first time in this context, including SRAM usage and Flash memory usage. These metrics are chosen carefully to match each RQ and to provide a clear understanding of the performance and effectiveness of the used LLMs for Arduino code generation.

To answer RQ1 (Overall Correctness), both syntactic and functional correctness [61] of the generated code were considered. Syntactic correctness is verified using the Arduino compiler, with default settings, to detect syntax mistakes, undeclared variables, missing semicolons, etc. while functional correctness is evaluated by executing the generated code and comparing its output with expected output. If the initial attempt generated correct code, it was marked successful; otherwise, it was marked failed.

To answer RQ2 (Code Performance), the performance of the generated code was compared with the reference code. The performance of each code was measured in terms of execution time/runtime [59] (i.e., the amount of time required to execute the code), Flash memory usage (i.e., the memory space required to store the generated code), and SRAM usage (i.e., the memory space required for the generated code to create and manipulate variables when it runs). The Arduino IDE was used to measure the amount of SRAM and Flash memory usages, while execution time was measured in microseconds. Furthermore, each experiment was repeated for two runs: the first run (automatically triggered through the Arduino IDE upload process) was treated as a warm-up and discarded to reduce measurement noise, while the second run was obtained by manually pressing the used board's reset button and used as the recorded measurement. Also, input/output operations (e.g., Serial.print()) were isolated from timing to ensure that only the core computation was measured. The execution time measurement was determined using the pseudo-code shown below. The execution time measurement code was added to all codes before execution to obtain execution time, Flash memory, usage and SRAM usage. Any code that executes faster, uses less Flash memory, and consumes less SRAM is considered the best and outperforms the others. However, not all codes perform well in all metrics. Therefore, each metric was compared individually.

Execution time measurement pseudo-code:

```
Begin
   start_time ← current_time()
   execute target_code
   end_time ← current_time()
   elapsed_time ← end_time − start_time
   print elapsed_time
End
```

To answer RQ3 (Multi-round Error Correction), the models were instructed to regenerate any code that did not pass the overall correctness in RQ1. The aim is to complete each task in as few attempts as possible. If the code generated by an LLM is incorrect, the second attempt is made using compiler error feedback or author feedback in case the output is incorrect, the process continues until either the number of attempts reaches the maximum of five, or a correct solution is obtained. Therefore, it leads to the #attempt metric [56], which represents the number of times the user prompted an LLM to generate a correct solution.

To answer RQ4 (Code Complexity), the complexity of the generated code was compared to the reference code. Code complexity is one of the key factors greatly affecting maintainability, readability, and overall code quality [77]. Hence, code quality cannot be evaluated only on correctness (RQ1) or performance (RQ2); it also requires understanding of the structural complexity and adherence to established coding standards [16]. For this RQ, the following metrics were used:

- Cyclomatic Complexity (CC) [75]: It measures the number of linearly independent paths in a program, which directly correlates with the number of decision points in the code. Lower CC is generally considered a sign of higher code quality and it is calculated using Equation 3.1.

$$CC = E - N + 2P \quad \text{..... (3.1)}$$

Where:

- E is the number of edges.
- N is the number of nodes.
- P is the number of connected components or exit points.

- Non-Comment Lines of Code (NCLOC)[62]: It measures the number of executable lines in a code, excluding comments and empty lines, which is often used to estimate development effort, cost, and productivity. Therefore, it supports quality analysis and maintainability assessment [78]. Also, codes with higher NCLOC are typically more complex and harder to maintain, while shorter codes improve readability and ease of testing.

It is worth mentioning that both CC and NCLOC metrics were computed using the Lizard tool provided by [79] and used by many researchers [17].

To answer RQ5 (Code Similarity), the generated code was compared to the reference code to derive additional information about the difference between both codes by employing the CodeBLEU, a widely adopted similarity metric [76]. It is a composite metric with the scores being the weighted average of 4 different sub-metrics treating code differently: n-gram

matching (BLEU), syntax match, data flow match, and semantic match [41]. CodeBLEU is calculated using Equation 3.2.

$$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{Syntax Match} + \gamma \cdot \text{Data Flow Match} + \delta \cdot \text{Semantic Match} \ldots. \quad (3.2)$$

Where:

- BLEU: measures n-gram overlap between generated and reference code.
- Syntax Match: compares Abstract Syntax Trees (AST).
- Data Flow Match: evaluates the consistency of variable usage and dependencies.
- Semantic Match: assesses code structure and functionality.
- The weights α, β, γ, δ are tunable hyperparameters, allowing flexibility based on the importance of each component in a specific context. In this thesis, equal weights ($\alpha = \beta = \gamma = \delta = 0.25$) were used which are the default values of the used tool that calculates this metric.

A value of 0 in CodeBLEU indicates no similarity between the generated and reference code, while a value closer to 1 indicates a high degree of similarity. This metric was computed using the CodeXGLUE tool provided by [80] and used by many researchers [34].

It is worth mentioning that for all conducted experiments, the same Arduino device, well-known original Arduino Uno Rev3[9], was used with the Arduino IDE specifications as specified in Table 3.1. The Arduino Uno was selected due to its widespread adoption in different applications and is a common device for conducting research in IoT as stated in [81].

Table 3.1: Arduino IDE Specifications.

	Specifications	**Detail**
Arduino IDE	IDE version	2.3.5
	GCC compiler version	7.3.0
	GCC compiler optimization level	Os (default).

[9] https://store.arduino.cc/products/arduino-uno-rev3

Chapter 4: Results and Discussion

4.1 Introduction

This section presents and discusses the experimental results of this thesis, where each of the identified RQ, summarized with a short title, is answered and discussed in the following sections based on the obtained findings, showing how each LLM performed across different evaluation metrics and offering a comprehensive comparison of their strengths and limitations.

4.2 Overall Correctness (RQ1)

The syntactic and functional correctness success rate of all generated codes was obtained in the first iteration. Figure 4.1 shows an impressive performance by ChatGPT-4o, achieving a remarkable 96.8% correctness rate. DeepSeek-V3 follows with a strong performance of 90.3%, while GitHub Copilot attains 87.1%. Also, Gemini 2.0 Flash and LLaMA-3 both achieve 80.6%. On the other hand, Claude 3.5 Sonnet shows the lowest performance among the evaluated models, with a correctness rate of 67.7%. When looking at the errors in the generated codes, most were functional/logical errors (e.g., incorrect outputs or included unnecessary infinite loops) indicating that the models are effective in generating syntactically correct code and rarely make syntax errors. One of the possible valid reasons for generating syntactically correct code is that LLMs are pattern learners and they have learned many syntax patterns from the training datasets.

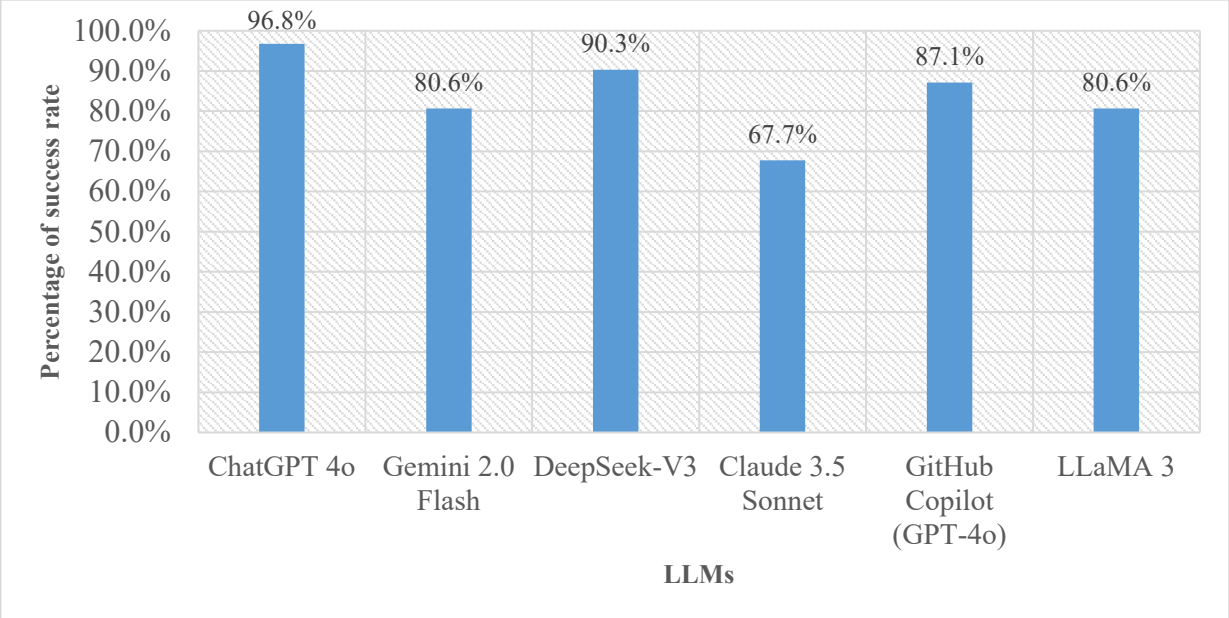

Figure 4.1: Overall correctness of LLM-generated code.

4.3 Code Performance (RQ2)

In this RQ, the aim is to investigate how the performance of code generated by LLMs compares to the reference code. Performance is a crucial factor in systems with limited resources and computing capabilities, such as Arduino based IoT and embedded systems. Hence, the generated and reference codes were compared based on three key performance metrics: execution time, Flash memory usage, and SRAM usage. Figure 4.2 compares execution time for all correct codes generated by LLMs in the first iteration. The execution time was categorized into three outcomes, as follows:

- Equal (blue bars): The generated code had the same execution time as the reference code.
- Greater (orange bars): The generated code had a longer execution time.
- Smaller (gray bars): The generated code had a shorter execution time.

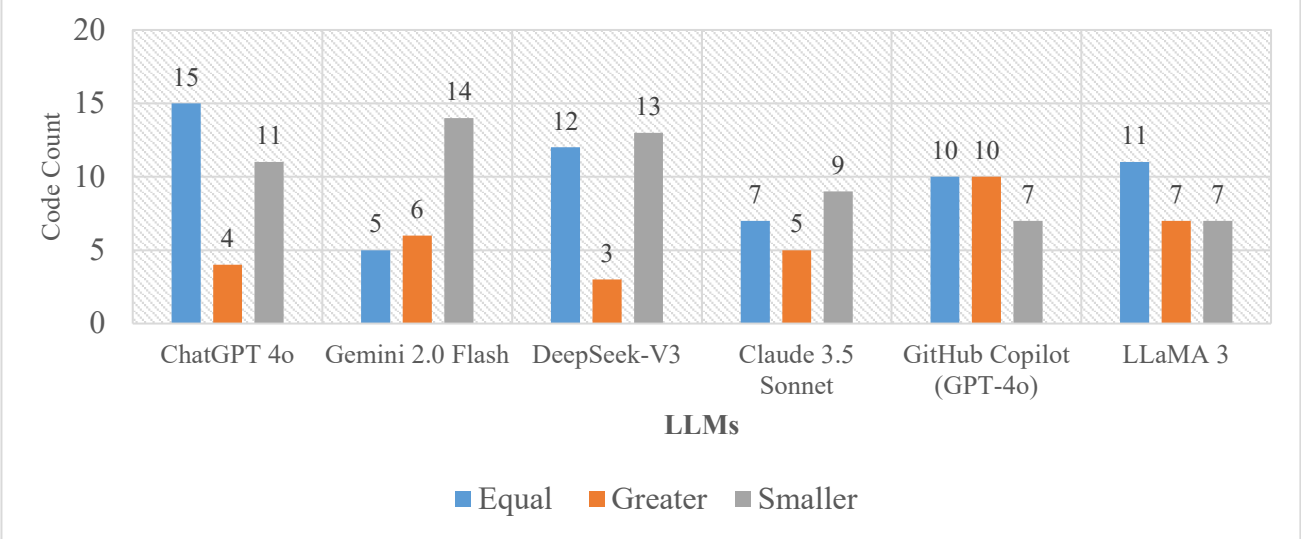

Figure 4.2: Execution time performance of LLM-generated code.

From Figure 4.2, it is clear that ChatGPT-4o shows the highest number of codes matching the execution times of the reference codes, while Gemini 2.0 Flash performed strongly, which shows a higher number of codes where execution time is smaller. Additionally, DeepSeek-V3 performed well with equal and smaller execution times. Interestingly, Claude 3.5 Sonnet shows a higher number of codes where execution time is smaller, but overall correctness (RQ1) is the poorest among the models. GitHub Copilot had an even distribution between equal and greater execution times, while LLaMA-3 presented a balanced number of greater and smaller codes.

Figure 4.3 shows the comparison of Flash memory usage for all generated codes in the first iteration, which are categorized into three outcomes: equal, greater, and smaller, similar to what was done for execution times comparison. The results show that DeepSeek-V3 has strong optimization capability with the highest number of codes with smaller Flash memory usage than the reference codes. In comparison, models such as Gemini 2 Flash and GitHub Copilot exhibited more codes with higher Flash memory usage. ChatGPT-4o presents a relatively balanced result, while LLaMA-3 maintains an even spread across the equal and greater categories, with a higher number of cases in the smaller Flash memory usage category.

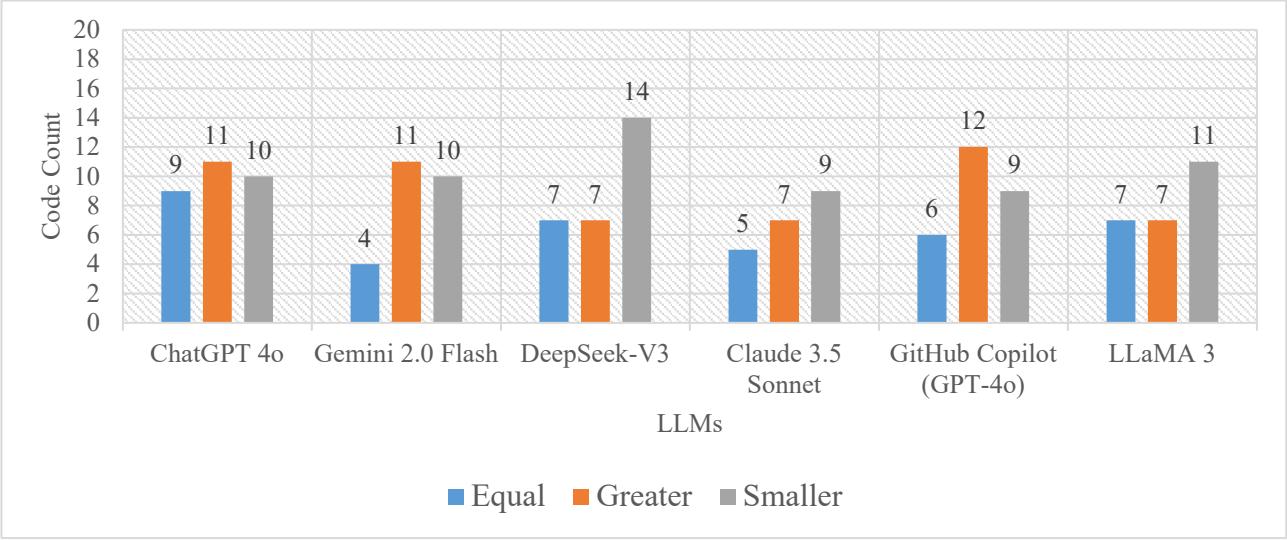

Figure 4.3: Flash memory usage performance of LLM-generated code.

Figure 4.4 compares SRAM usage across evaluated LLMs, categorizing outcomes also into equal, greater, or smaller usage. From the results, ChatGPT-4o demonstrates the highest number of codes where the generated code exhibits equal SRAM usage compared to the reference code, followed closely by DeepSeek-V3 and LLaMA-3. Also, GitHub Copilot performed well. In contrast, Gemini 2.0 Flash exhibits a more varied distribution with a higher number of greater SRAM usages, while Claude 3.5 Sonnet shows fewer codes of reduced SRAM usage compared to other models.

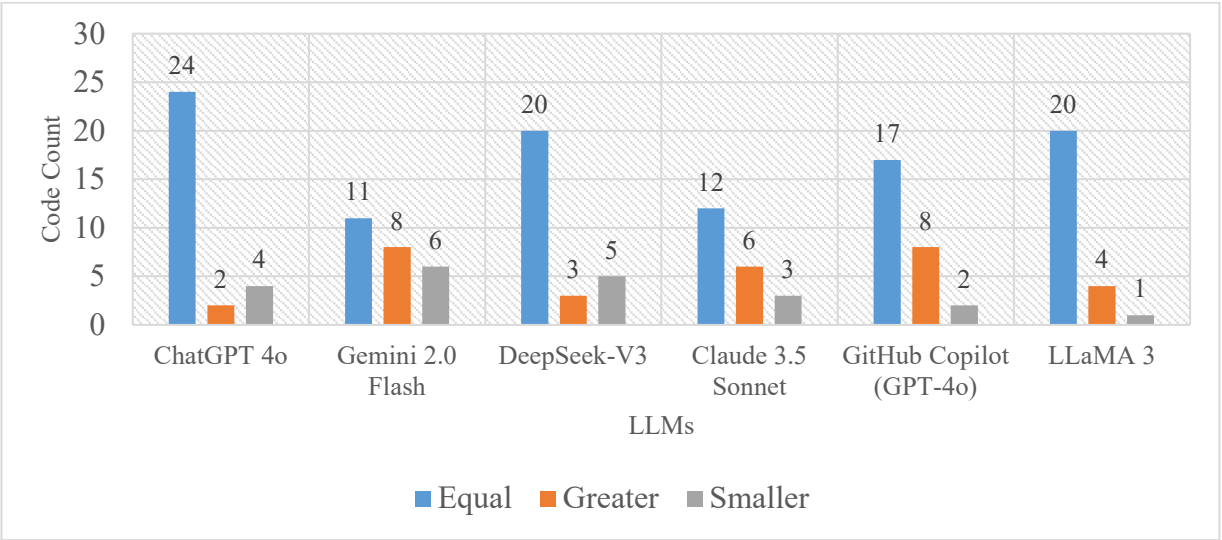

Figure 4.4: SRAM usage performance of LLM-generated code.

4.4 Multi-attempt Code Correction (RQ3)

This RQ examines the effectiveness of the multi-attempt correction process in enhancing code generation for functional correctness across all used LLMs. Since all LLMs support multi-attempt conversations, they were instructed to regenerate codes up to five times if they fail during the functional correctness test in RQ1.

As shown in Figure 4.5, all used LLMs reduced incorrect outputs over attempts. Claude 3.5 Sonnet started with the highest number of incorrect codes (i.e., 10 incorrect codes) but managed to reduce them to just one incorrect code by the second attempt, maintaining that level to the fifth attempt. Similarly, Gemini 2.0 Flash and LLaMA-3 showed consistent improvement, reaching a minimum of one incorrect code by the fifth attempt. The number of incorrect codes generated by GitHub Copilot dropped from four to one by the second attempt. The best performer in this context is ChatGPT-4o, where it generated only one incorrect code in the first attempt and by the second attempt it was corrected. Based on the obtained results, it is clear that multi-attempt code correction, especially within the first few attempts, is useful for improving code generation by reducing the number of incorrect generated codes.

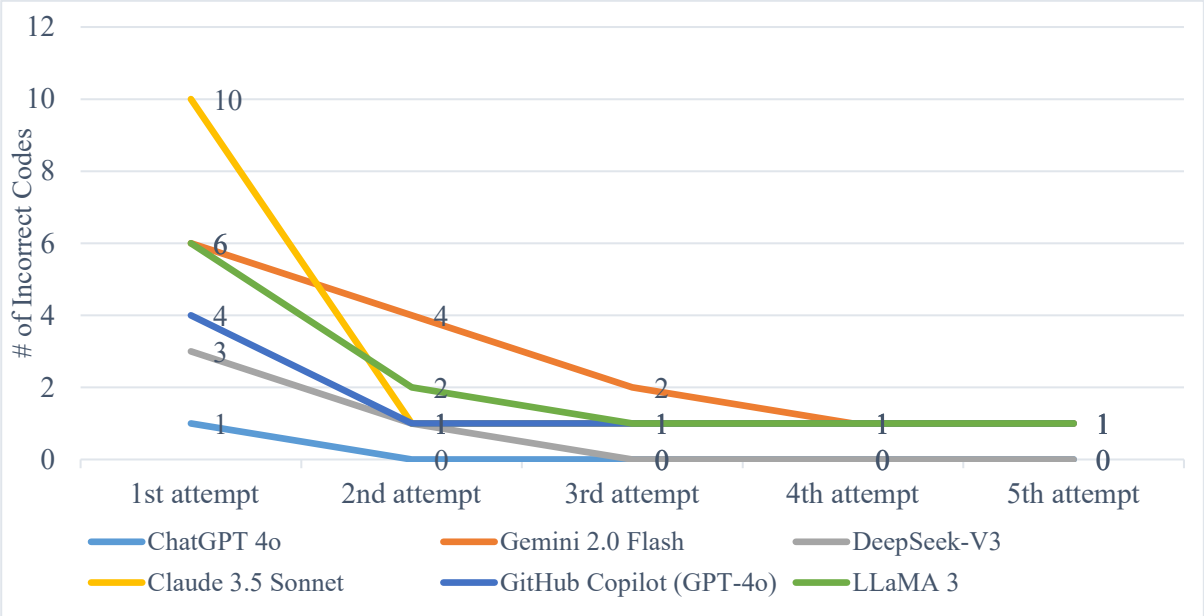

Figure 4.5: Multi-attempt code fixing progress across LLMs.

4.5 Code Complexity (RQ4)

Another important factor that notably affects code quality is code complexity. Highly complex code can be harder to understand and maintain. The results for CC are shown in Figure 4.6. The graphs demonstrate that most used LLMs tend to generate code with low CC (values of 2 and 3), similar to the reference codes. This shows that these LLMs generate relatively simple codes and maintainable logic structures. However, Gemini 2.0 Flash generated a higher number of codes with complexity levels of 4 and 5, which reduce code readability and maintainability. Additionally, a high CC can potentially lead to a high probability of errors and bugs.

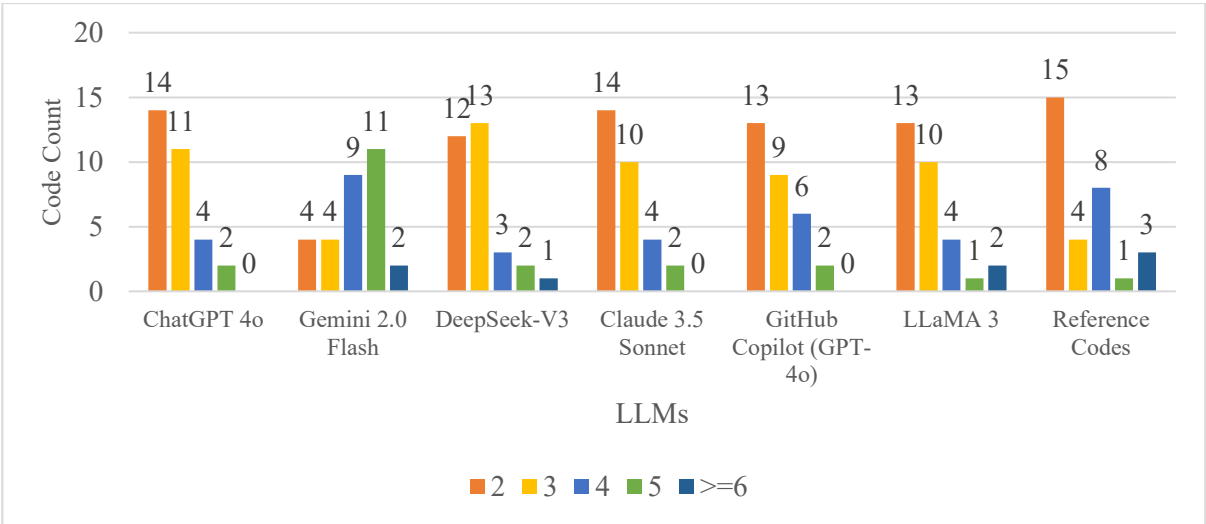

Figure 4.6: Cyclomatic complexity of LLM-generated code.

Figure 4.7 categorizes the NCLOC into five ranges. The results show that most used LLMs generated short codes in the 6-10 lines range, including ChatGPT-4o, DeepSeek-V3, and LLaMA-3. Claude 3.5 Sonnet and GitHub Copilot generated longer codes, with most in the 6-10 and 11-15 line ranges. However, Gemini 2.0 not only has more lines, but also higher CC values compared to other LLMs and the reference codes. Overall, these results highlight that while some LLMs align closely with the reference codes in terms of code length and complexity (i.e., ChatGPT-4o, DeepSeek-V3, and LLaMA-3), others vary significantly (i.e., Gemini 2.0 Flash, Claude 3.5 Sonnet and GitHub Copilot).

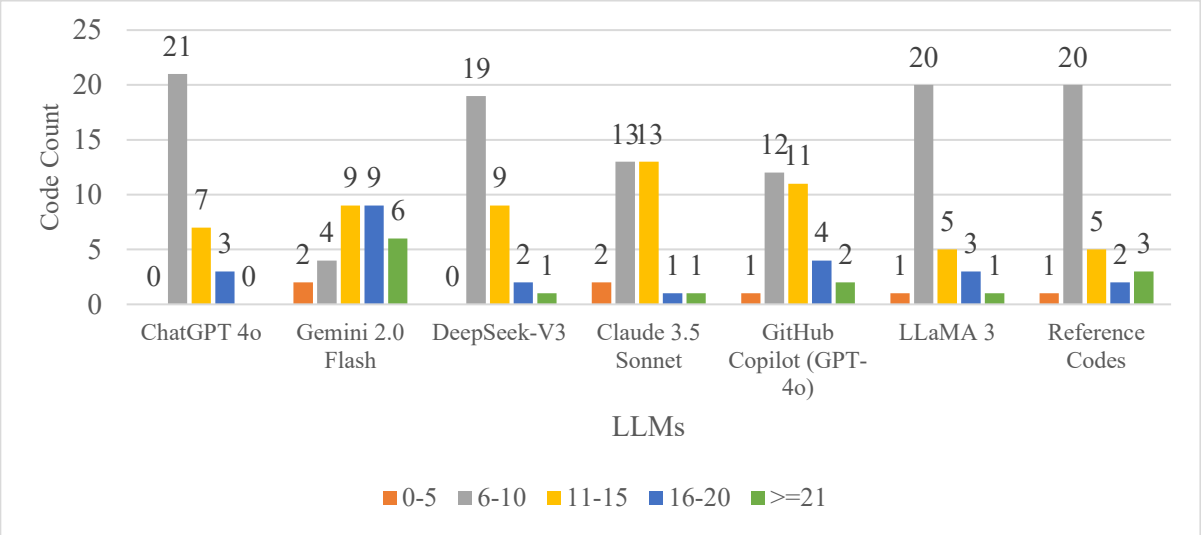

Figure 4.7: NCLOC of LLM-generated code.

4.6 Code Similarity (RQ5)

To derive additional information about the difference between the generated and reference codes, the CodeBLEU score (0-1 scale) was computed. Figure 4.8 shows the CodeBLEU scores for the reference and generated codes by all used LLMs. CodeBLEU scores show a significant variability in performance between the used LLMs, where ChatGPT-4o and LLaMA-3 achieved the highest median values, their output (i.e., generated codes) is similar to the reference codes. Gemini 2.0 Flash achieved the lowest median and a narrower interquartile range, which show a larger deviation from the reference codes. Moreover, DeepSeek-V3, Claude 3.5 Sonnet and GitHub Copilot performance was moderate and varied in consistency. While some models (i.e., ChatGPT-4o and LLaMA-3) successfully mimic human-like code patterns, others (i.e., Gemini 2.0 Flash) generate code with considerable structural or semantic deviation, which can significantly affect code quality and decrease readability and

maintainability.

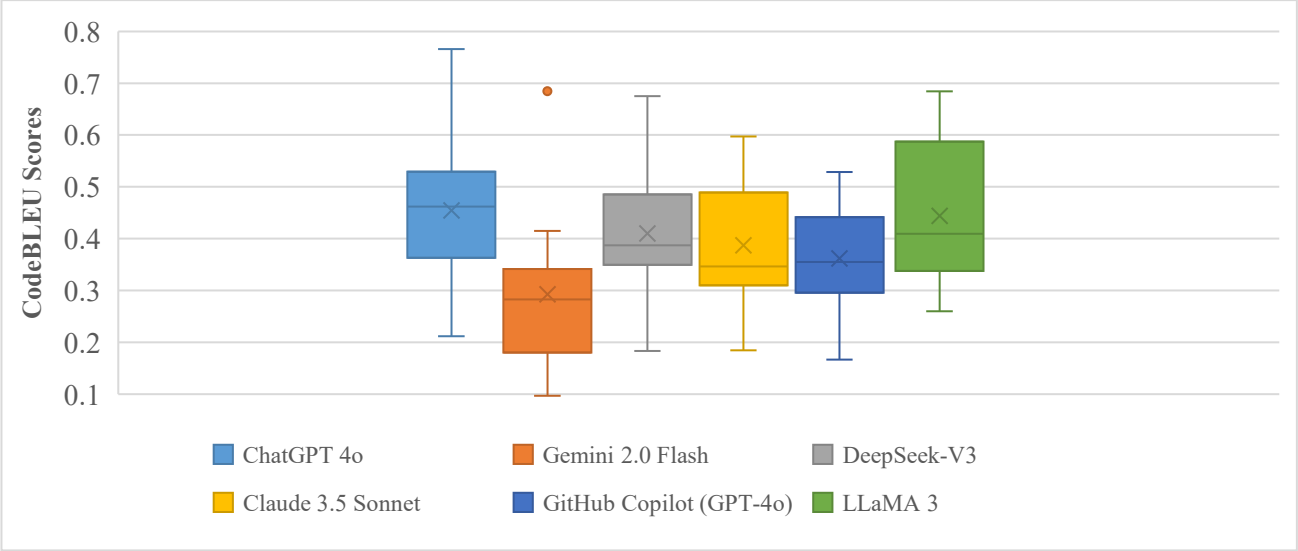

Figure 4.8: CodeBLEU scores of LLM-generated code.

4.7 Discussion

The evaluation of different LLMs for Arduino code generation reveals a trade-off between correctness, performance, and code quality. While ChatGPT-4o achieved the highest functional correctness rate (96.8%), its generated codes consistently have longer execution times than Gemini 2.0 Flash, meaning that there is no positive correlation between functional correctness and code efficiency. In contrast, DeepSeek-V3 has a functional correctness rate of 90.3%, less than ChatGPT-4o but delivered significantly lower execution times and reduced Flash memory consumption compared to ChatGPT-4o. Claude 3.5 Sonnet has the lowest functional correctness rate of 67.7%, but the error analysis revealed that many of its generated codes appended unnecessary infinite loops or the code was inside Arduino's continuously running loop(), even when the used prompt did not require it. Additionally, despite using the same prompt (i.e., zero-shot) with all used LLMs, Claude 3.5 Sonnet showed the least ability to interpret and adhere to the prompt constraints. GitHub Copilot achieved a functional correctness rate of 87.1%, ranking third behind ChatGPT-4o and DeepSeek-V3. Although Copilot was trained primarily on a large corpus of codes from GitHub and explicitly designed for programming tasks [82], it generated less efficient codes compared to other LLMs, which are trained on a broader range of datasets. This can be indicator that code-centric LLMs such as GitHub Copilot may not always generate the most efficient codes compared to general-purpose LLMs such as ChatGPT. This behavior arises for many possible reasons such as a

large portion of public GitHub codes where Copilot is trained on are written for readability and maintainability, not efficiency, and often are not written to target resource-constrained devices such as Arduino.

The effectiveness of multi-attempt error correction across all used LLMs highlights the potential of iterative refinement in practical software development. GitHub Copilot quickly dropped from four to one incorrect code by the second attempt, and all used LLMs corrected the majority of their errors by reaching the fifth attempt of corrections. These results suggest that integrating compiler or developer feedback into prompts can be a powerful strategy to improve LLM-based code generation.

Code complexity metrics further highlight the trade-off between maintainability and performance. ChatGPT-4o, LLaMA-3, and Claude 3.5 Sonnet consistently generated codes with CC values in the 2–3 range, comparable to the reference codes. This lower complexity enhances readability and reduces the cognitive load required for developers to understand and maintain code. In contrast, Gemini 2.0 Flash tended to generate codes with higher complexity (≥4) and longer lines, often due to optimizations such as loop unrolling or generating custom functions instead of using built-in functions or libraries. Although these programming strategies can improve execution speed, they also increase code complexity, raising concerns about code understandability and maintainability.

The CodeBLEU similarity analysis reinforces these findings by clearly highlighting differences in how closely the LLMs align with human coding practices. ChatGPT-4o and LLaMA-3 achieved the highest CodeBLEU scores, indicating strong structural and semantic similarity to the reference codes. This alignment translated not only into readable and maintainable codes but also into comparable performance and code complexity. In contrast, although Gemini 2.0 Flash produced longer and more complex codes with faster execution times, its substantially lower CodeBLEU scores reveal a significant difference from the reference codes.

In general, the results confirm that none of the LLMs excelled in all tasks. Table 4.1 shows an overall comparison between the used LLMs. These findings highlight the necessity of choosing the right LLMs depending on a program's priorities (e.g., correctness, performance, maintainability, or alignment with human coding practices).

Table 4.1: Summary of evaluting LLMs for Arduino code generation.

LLM	Overall Correctness (RQ1)	Execution Time (RQ2)	Flash Usage (RQ2)	SRAM Usage (RQ2)	Multi-Attempt Fixing (RQ3)	Cyclomatic Complexity (RQ4)	Code Length (NCLOC) (RQ4)	Code Similarity (RQ5)
ChatGPT-4o	**96.8% (Best)**	Mostly Equal	Balanced (Equal/ Greater /Smaller)	**Mostly Equal (Best)**	**Fixed All by 2nd Attempt**	**Low**	**Short**	**Most similar (Best)**
DeepSeek-V3	90.3%	Equal/Smaller	**Most Memory Efficient (Best)**	Mostly Equal	Fixed All by 3rd Attempt	**Low**	**Short**	Mostly Similar
GitHub Copilot	87.1%	Balanced (Equal/ Greater /Smaller)	Mostly Greater	Mostly Equal	Needed up to 2nd Attempt	Low/High	Medium	Less Similar
Gemini 2.0 Flash	80.6%	**Mostly Smaller (Best)**	Mostly Greater	Equal/Greater	Fixed Most by 5th Attempt	High	Longer	Least Similar
LLaMA-3	80.6%	Mostly Equal	Mostly Smaller	Mostly Equal	Fixed Most by 5th Attempt	**Low**	**Short**	Mostly Similar
Claude 3.5 Sonnet	67.7%	Equal/Smaller	Mostly Smaller	Mostly Equal	Fixed Most by 5th Attempt	**Low**	Short/ Medium	Less Similar

Chapter 5: Conclusions and Future Works

Chapter 5: Conclusions and Future Works

5.1 Conclusions

This thesis provides a comprehensive evaluation of LLMs for Arduino code generation, highlighting LLMs' strengths and limitations in producing efficient, maintainable, and human-like code. Six state-of-the-art LLMs, ChatGPT-4o, Gemini 2.0 Flash, DeepSeek-V3, Claude 3.5 Sonnet, GitHub Copilot, and LLaMA-3, were evaluated using a dataset of 31 Arduino programs. The evaluation was conducted across multiple dimensions, including functional correctness, execution time, memory usage (Flash and SRAM), code complexity, similarity to human-written code, and multi-round error correction. The experimental results of this thesis show that none of the evaluated LLMs excel across all evaluation dimensions. ChatGPT-4o leads in functional correctness and generated code that most closely to human-written code in terms of readability and similarity, while Gemini 2.0 Flash excels in runtime efficiency but often generated more complex code with lower similarity to reference code, and DeepSeek-V3 demonstrates strong potential for memory-optimized applications through efficient Flash memory usage. The thesis also identifies limitations in current LLMs, including prompt adherence issues in Claude 3.5 Sonnet and unexpected inefficiencies in GitHub Copilot despite its domain-specific trained LLM. Overall, the findings confirm that LLMs can serve as valuable tools for assisting Arduino based IoT development, but their suitability strongly depends on the specific application requirements. For instance, applications prioritizing correctness and readability can benefit more from ChatGPT-4o, whereas performance-critical applications can benefit from Gemini 2.0 Flash or DeepSeek-V3. This thesis contributes to a deeper understanding of capabilities of LLMs for code generation by providing practical insights for developers and researchers while also informing future improvements in LLM design and evaluation methodologies for IoT resource-constrained environments such as Arduino, a platform widely used in applications that impact many aspects of our daily life.

5.2 Future Works

For future works, this thesis can be extended to include evaluating other LLMs (including reasoning-capable LLMs) that are not used in this thesis, studying the impact of different prompt types on the quality of generated codes, evaluating other resource-constrained devices, such as Raspberry Pi, and using other programming languages that target hardware devices, such as MicroPython. Addressing these future research directions, can advance the potential of using LLMs as tools for generating efficient codes for IoT and embedded systems that are currently cover many aspects of daily life.

References

[1] H. Su, J. Ai, D. Yu, and H. Zhang, 'An Evaluation Method for Large Language Models' Code Generation Capability', in *Proceedings - 2023 10th International Conference on Dependable Systems and Their Applications, DSA 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 831–838. doi: 10.1109/DSA59317.2023.00118.

[2] W. Wang, H. Ning, G. Zhang, L. Liu, and Y. Wang, 'Rocks Coding, Not Development: A Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks', *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 699–721, Jul. 2024, doi: 10.1145/3643758.

[3] Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, 'No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT', *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1548–1584, Jun. 2024, doi: 10.1109/TSE.2024.3392499.

[4] D. Yan, Z. Gao, and Z. Liu, 'A Closer Look at Different Difficulty Levels Code Generation Abilities of ChatGPT', in *Proceedings - 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 1887–1898. doi: 10.1109/ASE56229.2023.00096.

[5] A. Nazir and Z. Wang, 'A comprehensive survey of ChatGPT: Advancements, applications, prospects, and challenges', Sep. 01, 2023, *KeAi Publishing Communications Ltd.* doi: 10.1016/j.metrad.2023.100022.

[6] X. Hou *et al.*, 'Large Language Models for Software Engineering: A Systematic Literature Review', *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, Dec. 2024, doi: 10.1145/3695988.

[7] C. E. A. Coello, M. N. Alimam, and R. Kouatly, 'Effectiveness of ChatGPT in Coding: A Comparative Analysis of Popular Large Language Models', *Digital*, vol. 4, no. 1, pp. 114–125, Mar. 2024, doi: 10.3390/digital4010005.

[8] N. J. Al-Khafaji and B. K. Majeed, 'Evaluating Large Language Models using Arabic Prompts to Generate Python Codes', in *4th International Conference on Emerging Smart Technologies and Applications, eSmarTA 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/eSmarTA62850.2024.10638877.

[9] G. S. Black, B. P. Rimal, and V. M. Vaidyan, ‘Balancing Security and Correctness in Code Generation: An Empirical Study on Commercial Large Language Models’, *IEEE Trans. Emerg. Top. Comput. Intell.*, pp. 1–12, Aug. 2024, doi: 10.1109/tetci.2024.3446695.

[10] J. Wang and Y. Chen, ‘A Review on Code Generation with LLMs: Application and Evaluation’, in *Proceedings - 2023 1st IEEE International Conference on Medical Artificial Intelligence, MedAI 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 284–289. doi: 10.1109/MedAI59581.2023.00044.

[11] A. Nayyar and V. Puri, ‘A review of Arduino board’s, Lilypad’s & Arduino shields’, in *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*, 2016, pp. 1485–1492.

[12] S.-M. Kim, Y. Choi, and J. Suh, ‘Applications of the Open-Source Hardware Arduino Platform in the Mining Industry: A Review’, *Applied Sciences*, 2020, doi: https://doi.org/10.3390/app10145018.

[13] M. Yusro, N. Guntoro, and R. Rikawarastuti, ‘Utilization of microcontroller technology using Arduino board for Internet of Things (a systematic review)’, in *AIP Conference Proceedings*, Jun. 2021, p. 60004. doi: 10.1063/5.0041705.

[14] K. Petersen, S. Vakkalanka, and L. Kuzniarz, ‘Guidelines for conducting systematic mapping studies in software engineering: An update’, *Inf. Softw. Technol.*, vol. 64, pp. 1–18, 2015, doi: https://doi.org/10.1016/j.infsof.2015.03.007.

[15] M. Chen *et al.*, *Evaluating Large Language Models Trained on Code*. arXiv preprint, 2021. doi: 10.48550/arXiv.2107.03374.

[16] A. Clark, D. Igbokwe, S. Ross, and M. F. Zibran, ‘A Quantitative Analysis of Quality and Consistency in AI-generated Code’, in *Proceedings - 2024 7th International Conference on Software and System Engineering, ICoSSE 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 37–41. doi: 10.1109/ICoSSE62619.2024.00014.

[17] N. Nikolaidis, K. Flamos, K. Gulati, D. Feitosa, A. Ampatzoglou, and A. Chatzigeorgiou, ‘A Comparison of the Effectiveness of ChatGPT and Co-Pilot for Generating Quality Python Code Solutions’, in *Proceedings - 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion, SANER-*

C 2024, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 93–101. doi: 10.1109/SANER-C62648.2024.00018.

[18] B. Xu *et al.*, 'Are We Ready to Embrace Generative AI for Software Q&A?', in *Proceedings - 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 1713–1717. doi: 10.1109/ASE56229.2023.00023.

[19] V. Majdinasab, M. J. Bishop, S. Rasheed, A. Moradidakhel, A. Tahir, and F. Khomh, 'Assessing the Security of GitHub Copilot's Generated Code-A Targeted Replication Study', in *Proceedings - 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 435–444. doi: 10.1109/SANER60148.2024.00051.

[20] Z. Zhao, J. Sun, C.-H. Cai, and Z. Wei, 'Code Generation Using Self-Interactive Assistant', Institute of Electrical and Electronics Engineers (IEEE), Aug. 2024, pp. 2347–2352. doi: 10.1109/compsac61105.2024.00377.

[21] H. Hajipour, K. Hassler, T. Holz, L. Schonherr, and M. Fritz, 'CodeLMSec Benchmark: Systematically Evaluating and Finding Security Vulnerabilities in Black-Box Code Language Models', in *Proceedings - IEEE Conference on Safe and Trustworthy Machine Learning, SaTML 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 684–709. doi: 10.1109/SaTML59370.2024.00040.

[22] H. Yu *et al.*, ' CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pretrained Models ', in *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)* , Los Alamitos, CA, USA: IEEE Computer Society, Apr. 2024, pp. 428–439. doi: 10.1145/3597503.3623322.

[23] P. Aggarwal *et al.*, 'CodeSift: An LLM-Based Reference-Less Framework for Automatic Code Validation', in *IEEE International Conference on Cloud Computing, CLOUD*, IEEE Computer Society, 2024, pp. 404–410. doi: 10.1109/CLOUD62652.2024.00052.

[24] L. Rai, S. Khatiwada, C. Deng, and F. Liu, 'Cross-Language Code Development with Generative AI: A Source-to-Source Translation Perspective', in *2024 IEEE 7th International Conference on Electronic Information and Communication Technology, ICEICT 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 562–565. doi: 10.1109/ICEICT61637.2024.10671366.

[25] X. Du *et al.*, ‘Evaluating Large Language Models in Class-Level Code Generation’, in *Proceedings - International Conference on Software Engineering*, IEEE Computer Society, 2024, pp. 982–994. doi: 10.1145/3597503.3639219.

[26] A. M. Dumitran, A. C. Badea, and S. G. Muscalu, ‘Evaluating the Performance of Large Language Models in Competitive Programming: A Multi-Year, Multi-Grade Analysis’, in *18th International Conference on INnovations in Intelligent SysTems and Applications, INISTA 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/INISTA62901.2024.10683837.

[27] M. MacEdo, Y. Tian, F. Cogo, and B. Adams, ‘Exploring the Impact of the Output Format on the Evaluation of Large Language Models for Code Translation’, in *Proceedings - 2024 IEEE/ACM 1st International Conference on AI Foundation Models and Software Engineering, FORGE 2024*, Association for Computing Machinery, Inc, Apr. 2024, pp. 57–68. doi: 10.1145/3650105.3652301.

[28] F. A. Sakib, S. H. Khan, and A. H. M. R. Karim, ‘Extending the Frontier of ChatGPT: Code Generation and Debugging’, in *2024 International Conference on Electrical, Computer and Energy Technologies (ICECET, Sydney, Australia, 2024*, Jul. 2024. doi: 10.1109/ICECET61485.2024.10698405.

[29] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, ‘How Secure is Code Generated by ChatGPT?’, in *Conference Proceedings - IEEE International Conference on Systems, Man and Cybernetics*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 2445–2451. doi: 10.1109/SMC53992.2023.10394237.

[30] Y. Feng, S. Vanam, M. Cherukupally, W. Zheng, M. Qiu, and H. Chen, ‘Investigating Code Generation Performance of ChatGPT with Crowdsourcing Social Data’, in *Proceedings - International Computer Software and Applications Conference*, IEEE Computer Society, 2023, pp. 876–885. doi: 10.1109/COMPSAC57700.2023.00117.

[31] C. Tony, M. Mutas, N. E. D. Ferreyra, and R. Scandariato, ‘LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations’, in *Proceedings - 2023 IEEE/ACM 20th International Conference on Mining Software Repositories, MSR 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 588–592. doi: 10.1109/MSR59073.2023.00084.

[32] M. L. Siddiq, L. Roney, J. Zhang, and J. C. S. Santos, ‘Quality Assessment of ChatGPT Generated Code and their Use by Developers’, in *Proceedings - 2024*

IEEE/ACM 21st International Conference on Mining Software Repositories, MSR 2024, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 152–156. doi: 10.1145/3643991.3645071.

[33] M. Geng *et al.*, 'Interpretation-based Code Summarization', in *IEEE International Conference on Program Comprehension*, IEEE Computer Society, 2023, pp. 113–124. doi: 10.1109/ICPC58990.2023.00026.

[34] X. Gu *et al.*, 'On the Effectiveness of Large Language Models in Domain-Specific Code Generation', *ACM Transactions on Software Engineering and Methodology*, Oct. 2024, doi: 10.1145/3697012.

[35] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, 'An Empirical Study of the Non-determinism of ChatGPT in Code Generation', *ACM Transactions on Software Engineering and Methodology*, Sep. 2024, doi: 10.1145/3697010.

[36] B. Kou, S. Chen, Z. Wang, L. Ma, and T. Zhang, 'Do Large Language Models Pay Similar Attention Like Human Programmers When Generating Code?', *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2261–2284, Jul. 2024, doi: 10.1145/3660807.

[37] Y. Dong, X. Jiang, Z. Jin, and G. Li, 'Self-collaboration Code Generation via ChatGPT', *ACM Transactions on Software Engineering and Methodology*, Sep. 2024, doi: 10.1145/3672459.

[38] X. Jiang *et al.*, 'Self-planning Code Generation with Large Language Models', *ACM Transactions on Software Engineering and Methodology*, Sep. 2024, doi: 10.1145/3672456.

[39] J. Li♂, G. Li, Y. Li, and Z. Jin, 'Structured Chain-of-Thought Prompting for Code Generation', *ACM Transactions on Software Engineering and Methodology*, Aug. 2024, doi: 10.1145/3690635.

[40] A. Koubaa, B. Qureshi, A. Ammar, Z. Khan, W. Boulila, and L. Ghouti, 'Humans are still better than ChatGPT: Case of the IEEEXtreme competition', *Heliyon*, vol. 9, no. 11, Nov. 2023, doi: 10.1016/j.heliyon.2023.e21624.

[41] M. Evtikhiev, E. Bogomolov, Y. Sokolov, and T. Bryksin, 'Out of the BLEU: How should we assess quality of the Code Generation models?', *Journal of Systems and Software*, vol. 203, p. 111741, 2023, doi: https://doi.org/10.1016/j.jss.2023.111741.

[42] V. Corso, L. Mariani, D. Micucci, and O. Riganelli, ‘Generating Java Methods: An Empirical Assessment of Four AI-Based Code Assistants’, in *IEEE International Conference on Program Comprehension*, IEEE Computer Society, 2024, pp. 13–23. doi: 10.1145/3643916.3644402.

[43] C. Liu *et al.*, ‘Guiding ChatGPT for Better Code Generation: An Empirical Study’, in *Proceedings - 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 102–113. doi: 10.1109/SANER60148.2024.00018.

[44] M. Guo, ‘Java Web Programming with ChatGPT’, in *2024 5th International Conference on Mechatronics Technology and Intelligent Manufacturing, ICMTIM 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 834–838. doi: 10.1109/ICMTIM62047.2024.10629560.

[45] D. G. Paul, H. Zhu, and I. Bayley, ‘ScenEval: A Benchmark for Scenario-Based Evaluation of Code Generation’, in *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*, IEEE, Jul. 2024, pp. 55–63. doi: 10.1109/AITest62860.2024.00015.

[46] J. Li, Y. Zhang, Z. Karas, C. Mcmillan, K. Leach, and Y. Huang, ‘Do Machines and Humans Focus on Similar Code? Exploring Explainability of Large Language Models in Code Summarization’, in *IEEE International Conference on Program Comprehension*, IEEE Computer Society, 2024, pp. 47–51. doi: 10.1145/3643916.3644434.

[47] Z. Yang *et al.*, ‘Exploring and Unleashing the Power of Large Language Models in Automated Code Translation’, *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1585–1608, Jul. 2024, doi: 10.1145/3660778.

[48] A. Bucaioni, H. Ekedahl, V. Helander, and P. T. Nguyen, ‘Programming with ChatGPT: How far can we go?’, *Machine Learning with Applications*, vol. 15, p. 100526, Mar. 2024, doi: 10.1016/j.mlwa.2024.100526.

[49] A. Rizvi, N. Simon, J. Tocho, A. Yongaci, S. Abi-Karam, and C. Hao, ‘Evaluating Large Language Models for High-Level Synthesis’, in *2024 IEEE Opportunity Research Scholars Symposium (ORSS)*, 2024, pp. 49–52. doi: 10.1109/ORSS62274.2024.10697938.

[50] D. de-Fitero-Dominguez, E. Garcia-Lopez, A. Garcia-Cabot, and J. J. Martinez-Herraiz, ‘Enhanced automated code vulnerability repair using large language models’, *Eng. Appl. Artif. Intell.*, vol. 138, Dec. 2024, doi: 10.1016/j.engappai.2024.109291.

[51] R. Afsharmazayejani, M. M. Shahmiri, P. Link, H. Pearce, and B. Tan, ‘Toward Hardware Security Benchmarking of LLMs’, in *2024 IEEE LLM Aided Design Workshop, LAD 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/LAD62341.2024.10691745.

[52] F. R. Kashanaki, M. Zakharov, and J. Renau, ‘HDLEval Benchmarking LLMs for multiple HDLs’, in *2024 IEEE LLM Aided Design Workshop, LAD 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/LAD62341.2024.10691770.

[53] M. Liu, N. Pinckney, B. Khailany, and H. Ren, ‘Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation’, in *IEEE/ACM International Conference on Computer-Aided Design, Digest of Technical Papers, ICCAD*, Institute of Electrical and Electronics Engineers Inc., 2023. doi: 10.1109/ICCAD57390.2023.10323812.

[54] K. Moratis, T. Diamantopoulos, D. N. Nastos, and A. Symeonidis, ‘Write me this Code: An Analysis of ChatGPT Quality for Producing Source Code’, in *Proceedings - 2024 IEEE/ACM 21st International Conference on Mining Software Repositories, MSR 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 147–151. doi: 10.1145/3643991.3645070.

[55] P. Vijayaraghavan *et al.*, ‘VHDL-Eval: A Framework for Evaluating Large Language Models in VHDL Code Generation’, in *2024 IEEE LLM Aided Design Workshop (LAD)*, IEEE, Jun. 2024, pp. 1–6. doi: 10.1109/LAD62341.2024.10691836.

[56] T. Miah and H. Zhu, ‘User Centric Evaluation of Code Generation Tools (Invited Paper)’, in *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*, 2024, pp. 109–119. doi: 10.1109/AITest62860.2024.00022.

[57] N. Petrovic, S. Konicanin, and S. Suljovic, ‘ChatGPT in IoT Systems: Arduino Case Studies’, in *2023 IEEE 33rd International Conference on Microelectronics, MIEL 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 1–4. doi: 10.1109/MIEL58498.2023.10315791.

[58] D. Cotroneo, A. Foggia, C. Improta, P. Liguori, and R. Natella, ‘Automating the correctness assessment of AI-generated code for security contexts’, *Journal of Systems and Software*, vol. 216, Oct. 2024, doi: 10.1016/j.jss.2024.112113.

[59] C. Niu, T. Zhang, C. Li, B. Luo, and V. Ng, ‘On Evaluating the Efficiency of Source Code Generated by LLMs’, in *Proceedings - 2024 IEEE/ACM 1st International Conference on AI Foundation Models and Software Engineering, FORGE 2024*, Association for Computing Machinery, Inc, Apr. 2024, pp. 103–107. doi: 10.1145/3650105.3652295.

[60] C. Niu, C. Li, V. Ng, D. Chen, J. Ge, and B. Luo, ‘An Empirical Comparison of Pre-Trained Models of Source Code’, in *Proceedings - International Conference on Software Engineering*, IEEE Computer Society, 2023, pp. 2136–2148. doi: 10.1109/ICSE48619.2023.00180.

[61] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. (Jack) Jiang, ‘GitHub Copilot AI pair programmer: Asset or Liability?’, *J. Syst. Softw.*, vol. 203, no. C, Sep. 2023, doi: 10.1016/j.jss.2023.111734.

[62] L. Beurer-Kellner, M. Fischer, and M. Vechev, ‘Prompting Is Programming: A Query Language for Large Language Models’, *Proceedings of the ACM on Programming Languages*, vol. 7, Jun. 2023, doi: 10.1145/3591300.

[63] K. Jin, C. Y. Wang, H. V. Pham, and H. Hemmati, ‘Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation’, in *Proceedings - 2024 IEEE/ACM 21st International Conference on Mining Software Repositories, MSR 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 167–171. doi: 10.1145/3643991.3645074.

[64] R. Khojah, M. Mohamad, P. Leitner, and F. G. de Oliveira Neto, ‘Beyond Code Generation: An Observational Study of ChatGPT Usage in Software Engineering Practice’, *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1819–1840, Jul. 2024, doi: 10.1145/3660788.

[65] W. Mendes, S. Souza, and C. R. B. De Souza, ‘“You’re on a bicycle with a little motor”: Benefits and Challenges of Using AI Code Assistants’, in *2024 IEEE/ACM 17th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE)*, 2024, pp. 144–152.

[66] K. Jesse, T. Ahmed, P. T. Devanbu, and E. Morgan, ‘Large Language Models and Simple, Stupid Bugs’, in *Proceedings - 2023 IEEE/ACM 20th International Conference on Mining Software Repositories, MSR 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 563–575. doi: 10.1109/MSR59073.2023.00082.

[67] S. Hamer, M. D’Amorim, and L. Williams, ‘Just another copy and paste? Comparing the security vulnerabilities of ChatGPT generated code and StackOverflow answers’, in *Proceedings - 45th IEEE Symposium on Security and Privacy Workshops, SPW 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 87–94. doi: 10.1109/SPW63631.2024.00014.

[68] T. Xiao, C. Treude, H. Hata, and K. Matsumoto, ‘DevGPT: Studying Developer-ChatGPT Conversations’, in *Proceedings of the 21st International Conference on Mining Software Repositories*, in MSR ’24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 227–230. doi: 10.1145/3643991.3648400.

[69] M. DeLorenzo, V. Gohil, and J. Rajendran, ‘CreativEval: Evaluating Creativity of LLM-Based Hardware Code Generation’, in *2024 IEEE LLM Aided Design Workshop (LAD), San Jose, CA, USA*, Apr. 2024, pp. 1–5. doi: 10.1109/LAD62341.2024.10691798.

[70] D. G. Paul, H. Zhu, and I. Bayley, ‘Benchmarks and Metrics for Evaluations of Code Generation: A Critical Review’, in *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*, IEEE, Jul. 2024, pp. 87–94. doi: 10.1109/AITest62860.2024.00019.

[71] D. Palla and A. Slaby, ‘Evaluation of Generative AI Models in Python Code Generation: A Comparative Study’, *IEEE Access*, vol. 13, pp. 65334–65347, 2025, doi: 10.1109/ACCESS.2025.3560244.

[72] I. Kok, O. Demirci, and S. Ozdemir, ‘ When IoT Meet LLMs: Applications and Challenges ’, in *2024 IEEE International Conference on Big Data (BigData)* , Los Alamitos, CA, USA: IEEE Computer Society, Dec. 2024, pp. 7075–7084. doi: 10.1109/BigData62323.2024.10825187.

[73] U. A. Shuvo, S. A. Dip, N. R. Vaskar, and A. B. M. A. Al Islam, ‘Assessing ChatGPT’s Code Generation Capabilities with Short vs Long Context Programming Problems’, in *Proceedings of the 2024 11th International Conference on Networking,*

Systems and Security, NSysS 2024, Association for Computing Machinery, Inc, Jan. 2025, pp. 32–40. doi: 10.1145/3704522.3704535.

[74] S. Mirjalili, A. A. Abdulla, B. A. Hassan, and T. A. Rashid, 'LLaMA-Adapter + MRP: Integrating Meta-Reasoning Prompting with LLaMA-Adapter for Efficient Multi-Modal and Task-Adaptive Reasoning', *TechRxiv preprint*, Jun. 2025, doi: 10.36227/techrxiv.175021696.61801668/v1.

[75] C. Ebert, J. Cain, G. Antoniol, S. Counsell, and P. Laplante, 'Cyclomatic Complexity', *IEEE Softw.*, vol. 33, no. 6, pp. 27–29, 2016, doi: 10.1109/MS.2016.147.

[76] S. Ren *et al.*, 'CodeBLEU: a Method for Automatic Evaluation of Code Synthesis', 2020, *arxiv*. doi: https://doi.org/10.48550/arXiv.2009.10297.

[77] Y. Tashtoush, N. Abu-El-Rub, O. Darwish, S. Al-Eidi, D. Darweesh, and O. Karajeh, 'A Notional Understanding of the Relationship between Code Readability and Software Complexity', *Information (Switzerland)*, vol. 14, no. 2, Feb. 2023, doi: 10.3390/info14020081.

[78] A. S. Nuñez-Varela, H. G. Pérez-Gonzalez, F. E. Martínez-Perez, and C. Soubervielle-Montalvo, 'Source code metrics: A systematic mapping study', *Journal of Systems and Software*, vol. 128, pp. 164–197, 2017, doi: https://doi.org/10.1016/j.jss.2017.03.044.

[79] T. Yin, 'Lizard: A simple code complexity analyser without caring about the c/c++ header files or java imports, supports most of the popular languages, pp. 21–7, 2020', 2024. [Online]. Available: https://github.com/terryyin/lizard

[80] S. Lu *et al.*, 'CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation', May 2021. doi: 10.48550/arXiv.2102.04664.

[81] Q. I. Sarhan, 'Systematic Survey on Smart Home Safety and Security Systems Using the Arduino Platform', 2020, *Institute of Electrical and Electronics Engineers Inc.* doi: 10.1109/ACCESS.2020.3008610.

[82] J.-H. R. Jiang, F. Wang, J. Shen, S.-J. Kim, and S. Kim, 'A Survey on Large Language Models for Code Generation', *ArXiv*, Jun. 2024, doi: https://doi.org/10.1145/3747588.

الخلاصة:

نماذج اللغة الكبيرة (LLMs) ، المعروفة أيضاً باسم الذكاء الاصطناعي التوليدي، أحدثت تحولاً في توليد الشيفرة البرمجية من خلال تحويل المطالبات المكتوبة باللغة الطبيعية إلى شيفرة قابلة للتنفيذ. ومع ذلك، فإن قدراتها في توليد الشيفرة للأجهزة ذات الموارد المحدودة، مثل أردوينو، المستخدمة في إنترنت الأشياء (IoT) والأنظمة المدمجة، ما تزال غير مستكشفة بشكل كافٍ. تقيّم هذه الأطروحة ستة من أحدث نماذج اللغة الكبيرة (ChatGPT-4o ، Gemini 2.0 Flash، DeepSeek-V3، Claude 3.5 Sonnet، GitHub Copilot، LLaMA-3) من حيث قدرتها على توليد شيفرة أردوينو صحيحة وفعّالة وعالية الجودة. وقد أُجري التقييم عبر ستة أبعاد، وهي: الصحة الوظيفية، وكفاءة زمن التشغيل، واستخدام الذاكرة، وجودة الشيفرة، ومدى التشابه مع الشيفرة التي يكتبها البشر، وتصحيح الأخطاء عبر جولات متعددة. تكشف النتائج أن ChatGPT- 4o يحقق أعلى مستوى من الصحة الوظيفية ، ويتوافق بدرجة كبيرة مع الشيفرة البشرية من حيث القابلية للقراءة والتشابه. أما Gemini 2.0 Flash فيولّد شيفرات تعمل بسرعة أكبر، إلا أن شيفراته تتسم بدرجة تعقيد عالية وتشابه منخفض مقارنة بالشيفرات المرجعية. وتظهر شيفرات DeepSeek-V3 أداءً جيداً من حيث تحسين استخدام ذاكرة الفلاش. بينما يواجه Claude 3.5 Sonnet بعض التحديات في فهم المطالبات. كما تتحسن صحة الشيفرة المولَّدة من جميع النماذج المستخدمة عند تطبيق تصحيح الأخطاء متعدد الجولات. بشكل عام، تشير النتائج إلى أنه لا يوجد نموذج واحد يتفوق في جميع جوانب التقييم، أي أن كل نموذج يتميز في جانب مختلف من جوانب توليد الشيفرة. لذلك، وبحسب أولويات البرنامج، ينبغي اختيار النموذج الأنسب؛ فعلى سبيل المثال، يتفوق ChatGPT-4o في الصحة الوظيفية، و Gemini 2.0 Flash في زمن التنفيذ، و DeepSeek-V3 في كفاءة الذاكرة. تسهم هذه النتائج في تطوير نماذج اللغة الكبيرة من خلال تحديد نقاط القوة والضعف لكل نموذج في توليد شيفرة أردوينو، حيث تُعد أردوينو منصة واسعة الاستخدام في إنترنت الأشياء والأنظمة المدمجة.

الكلمات المفتاحية: نماذج اللغة الكبيرة، أردوينو، توليد الشيفرة، إنترنت الأشياء، أداء الشيفرة.

حكومة إقليم كوردستان – العراق
وزارة التعليم العالي والبحث العلمي
جامعة دهوك
كلية العلوم
قسم علوم الحاسبات

تقييم نماذج اللغات الكبيرة لتوليد الكود البرمجي

رساله ماجستير مقدمة الى مجلس كلية العلوم في جامعة دهوك إستكمالا لمتطلبات الحصول على درجة الماجستير في علوم الحاسبات

مقدمة من قبل الباحث
سردار كوكز جبرو علي
بكالوريوس في علوم الحاسوب، جامعة دهوك – 2019

بإشراف
أ.م.د قصى ادريس سرحان

دهوك، إقليم كوردستان، جمهورية العراق

٢٧٢٦ ك ٢٠٢٦ م ١٤٤٧ هـ

پوختە

مودێلێن مەزن یێن زمانان (LLMs) کو ب ناڤێ ژیریا دەستکرد یا چێکەر (Generative AI) ژی دهێنە نیاسین، شۆڕشەک د چێکرنا کۆدان دا جێکریە ب رێکا گوهۆرینا داخوازان ب زمانێ سروشتی بۆ کۆدێن دروست. لێ دیسان، شیانێن وان د چێکرنا کۆدا بۆ ئامیرێن خودان ژێدەرێن سنووردار، وەک ئاردوینۆ (Arduino) کو د ئینتەرنێتا تشتان (IoT) و سیستمێن ئەمبێدد (Embedded Systems) دا دهێنە بکارئینان ب کێمی هاتینە ڤەکۆلین. ئەڤ تێزە هەلسەنگاندنێ بۆ شەش مودێلێن پێشکەفتی یێن زمانان (ChatGPT-4o، Gemini 2.0 Flash، DeepSeek-V3، Claude 3.5 Sonnet، GitHub Copilot، LLaMA-3) ژ بۆ چێکرنا کۆدێن دروست، کارا، و کوالیتیا بلند بۆ ئاردوینۆیێ. ئەڤ هەلسەنگاندنە ل سەر شەش ڕەهەندان هاتە ئەنجامدان، کو ئەڤانە: دروستییا کۆدی، کارایییا کۆدیددەمێ کارپێکرنێ، بکارئینانا میمۆریێ، کوالیتیا کۆدی، نێزیکی ژ کۆدێ مرۆڤ نڤیسای، و ڕاستکرنا کۆدی د چەند گەڕیان دا. ئەنجام دیار دکەن کو ChatGPT-4o بلندترین ئاستێ دروستییا کۆدی ب دەست ڤە ئینایە و د خواندن و نێزیکیێ دا گەلەک نێزیکی کۆدێ مرۆڤ نڤیسایە. Gemini 2.0 Flash کۆدێن ب لەزتر چێدکەت، لێ کۆدێن وێ ئالۆزییەکا بلند و نێزیکییەکا کێم دگەل کۆدێن مرۆڤ نڤیسایە هەیە. ئەو کۆدێن ژ لایێ DeepSeek-V3 ڤە دهێنە چێکرن، ژ لایێ پاشکەفتکرنا میمۆریا فلاش (Flash memory optimization) ڤە د باشن Claude 3.5 Sonnet. هندەک کێشە د تێگەهشتنا داخوازان دا هەبوون. دروستییا کۆدێن هەمی مودێلان باشتر لێ هات دەما کو ڕاستکرنا شێتییان د چەند گەڕیان دا هاتە بکارئینان. ب گشتی، ئەنجامێن ب دەست کەفتین نیشان ددەن کو چو مودێلەک ب تنێ د هەمی هەلسەنگاندنان دا باشترین نینە. ب واتایەکا دی، هەر مودێلەک د لایەنەکێ چێکرنا کۆدی دا باشترە. ژ بەر هندێ، بۆ بەرنامەیێن خودان ئەولەویەتێن جودا، دڤێت مودێلێن جودا بهێنە هەلبژارتن. بۆ نموونە ChatGPT-4o د دروستییا کارکەفتنێ دا یێ نایابە، د دەمەکێ دا Gemini 2.0 Flash د دەمێ جێبەجێکرنێ دا، و DeepSeek-V3 د کارایییا میمۆریێ دا باشترینە. ئەڤ دیتنە پشکداریێ د پێشئێخستنا بەردەوام یا مودێلێن زمان دا دکەن ب رێکا دەستنیشانکرنا خالێن بهێز و لاواز یێن هەر مودێلەکێ د چێکرنا کۆدێ ئاردوینۆیێ دا، کو ئاردوینۆ پلاتفۆرمەکێ بەربەلاڤە بۆ د ئینتەرنێتا تشتان (IoT) و سیستمێن ئەمبێدد دا.

پەیڤێن سەرەکی: مودێلێن مەزن یێن زمانان، ئاردوینۆ، چێکرنا کۆدی، ئینتەرنێتا تشتان (IoT)، ئەنجامدانا کۆدی.

حکومەتا هەرێما کوردستانا عیراقێ
وەزارەتا خواندنا بلند و ڤەکۆلینێن زانستی
زانکویا دهۆك
کولیژا زانست
پشکا زانستێن کومبیوتەری

هەلسەنگاندنا مودێلێن مەزن ییّن زمانان بۆ چێکرنا کۆدا

نامەیەکا پێشکێشکریە بۆ جڤاتا کولیژا زانست ل زانکۆیا دهۆک وەک پشکەک ژ پێداویستیێن بدەستڤەئینانا پلا ماستەر ب زانستێن کومپیوتەری

یا قوتابی
سردار کوکز جبرو علي

بکالوریوس ب زانستێن کومپیوتەری ل زانکۆیا دهۆک – 2019

سەرپەرشت
پ. هـ. د. قصی ادریس سرحان

دهۆک – هەرێما کوردستانێ - عیراق

٢٧٢٦ کوردی **٢٠٢٦ زاینی** **١٤٤٧ کۆچی**